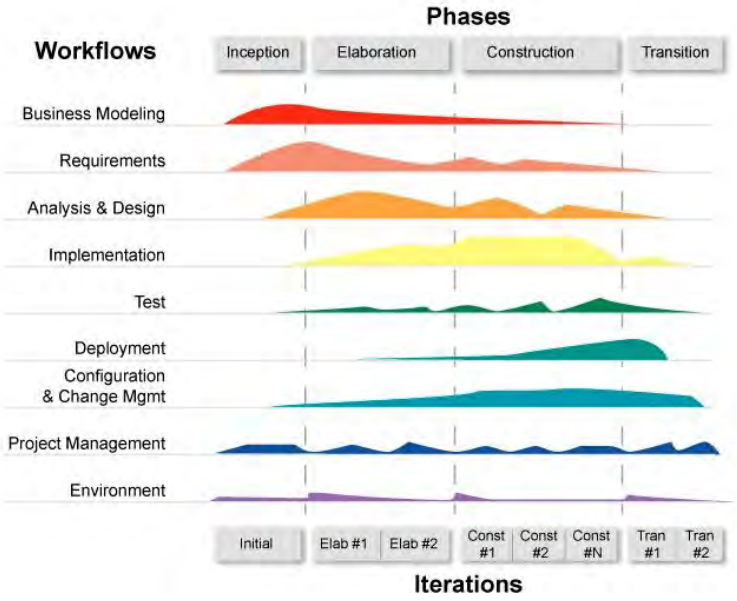
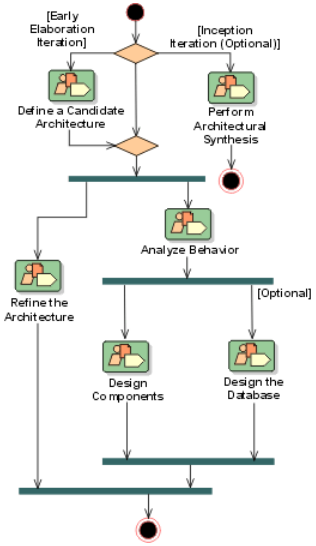
ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τεχνολογίες Υπηρεσιών Λογισμικού

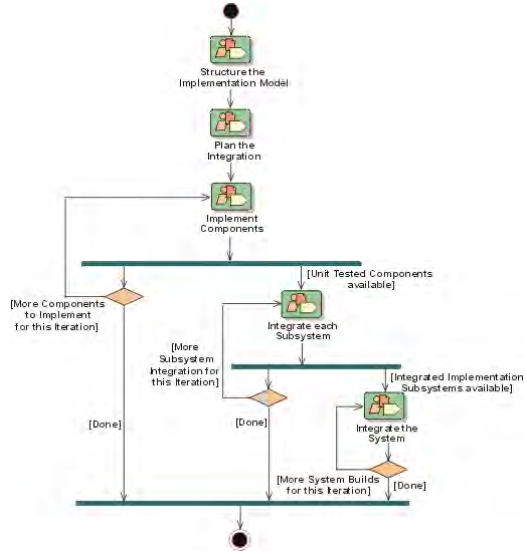
Τεχνολογίες Υπηρεσιών Λογισμικού

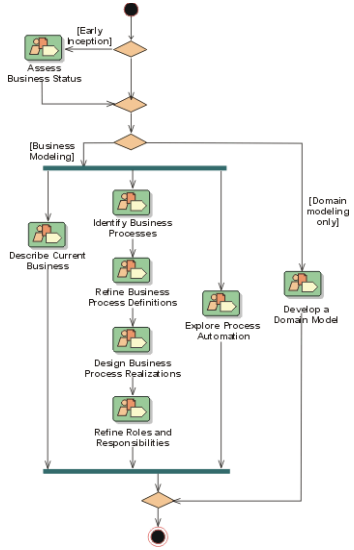
Β.Βεσκούκης, Καθ.ΕΜΠ, Π.Τσανάκας, Καθ.ΕΜΠ, Μ.Κόνιαρης, ΕΔΙΠ

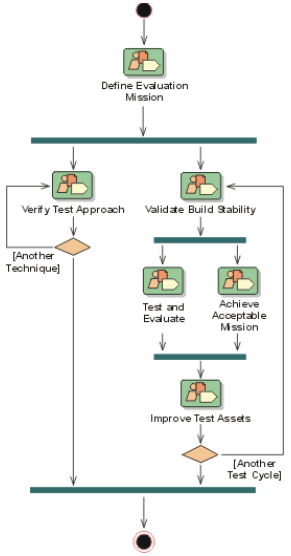
Software engineering

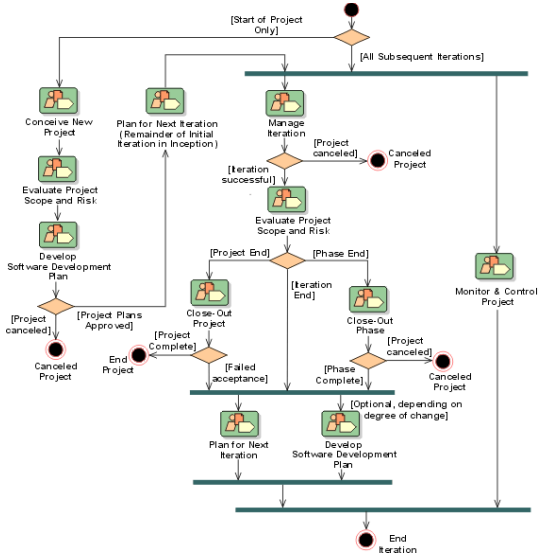




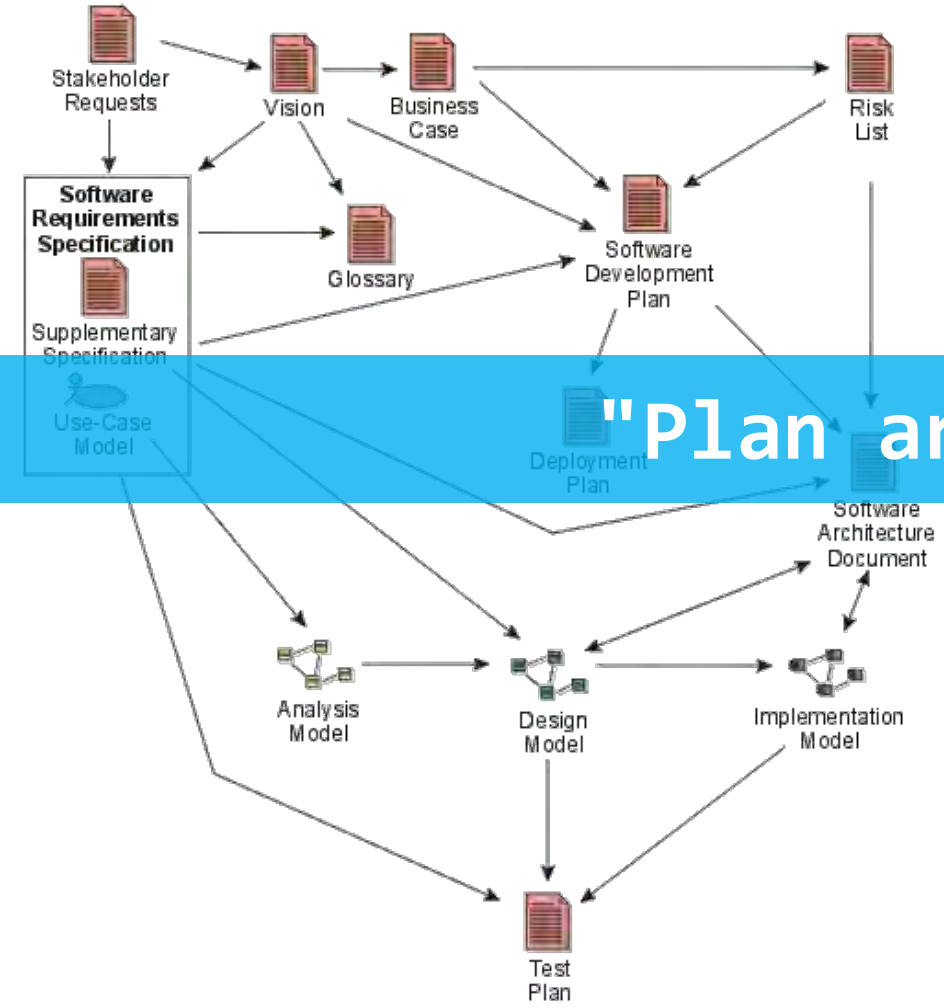




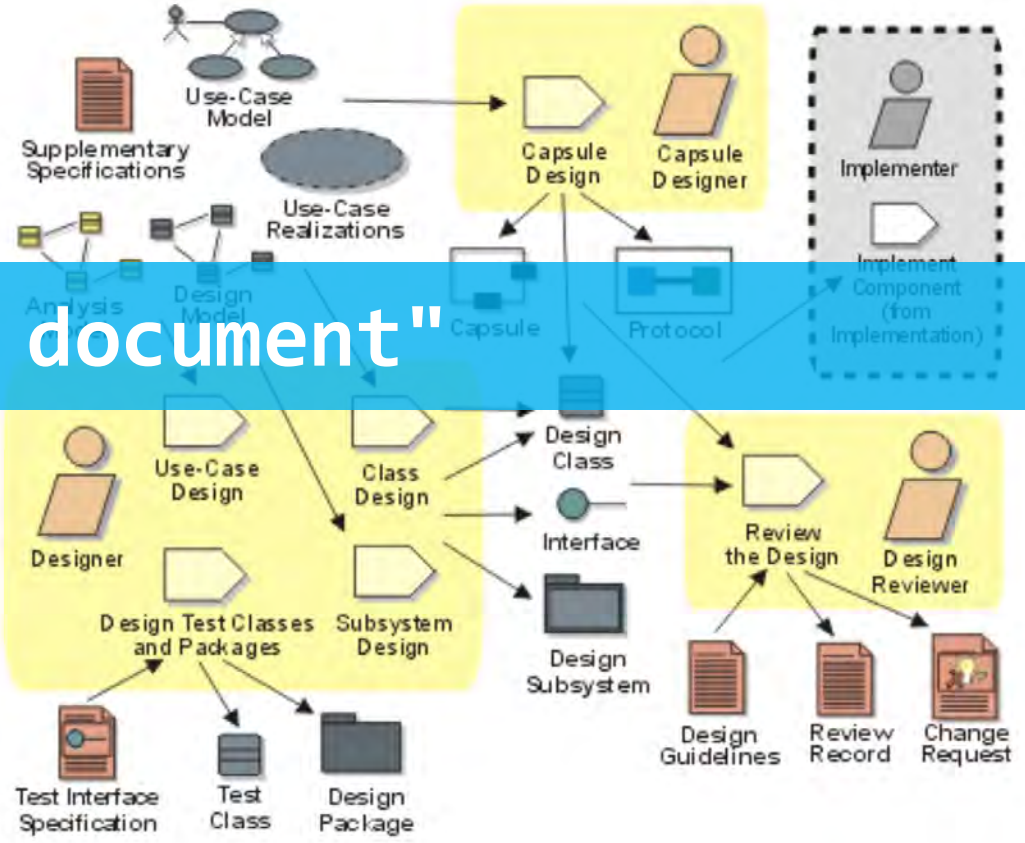




Documentation…



"Plan and document"



Some fair criticism on "Plan & Document"

Customers change their minds, (almost) always!

What to do

- Include some prototyping in the life cycle
- Align with business processes, deal with risk
- Keep fair useful documentation

What NOT to do

- Start and see how/where it goes
- Produce bad solutions to improve in the future
- Skip documentation altogether

SaaS: identifying characteristics

Software-as-a-Service needs to:

- Run on the cloud (server(s))
- Be accessed via browser and/or mobile (client(s))
- Perform as needed
- Satisfy availability, security, privacy, etc. non-functional requirements
- Invent new pricing models

SaaS *can be* more well-suited for Agile software development methodologies

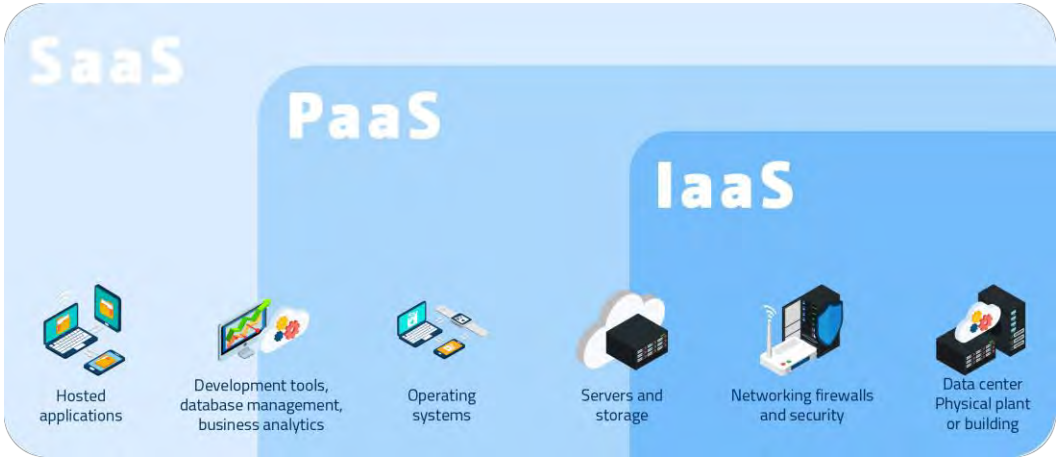
SaaS advantages

- No local installation => no hw compatibility needed
- Associated service data is kept within service (no back up, data loss concerns)
- Easier for groups to collaborate on same data
- Large, frequently updated data more wieldy if centralized + accessed remotely
- Developers maintain single copy of server software
- Upgrades don’t interfere with user experience
- Encourages more competition => better features

SaaS and Cloud Computing

SaaS has 3 requirements on IT infrastructure

- **Communication**: Allow any customer to interact with service
- **Availability**: Service + Communication vehicle continuously available
- **Scalability**: Infrastructure capable of dealing with demand fluctuations + spikes



SaaS: Runs on the cloud

Data is not stored locally

- More resilient to HW (disks) failures
- More easy to share and collaborate

No need for co-existing app versions

- For different operating systems or other compatibility requirements
- Easily distribute, maintain and upgrade your app
- Continuous feedback + request for changes possible at real-time

SaaS: Runs on the cloud

The cloud: (mostly) ordinary computers, switches, etc. resources

- Cheap, Scalable, Dependable (if redundant!)
- Professionally and effectively maintained by others
- HW abstractions (VMs, Docker, Kubernetes..)
- Pay for what you use, scale up/down instantly
- "Public/utility computing": Amazon, Google, MS

- Ground for everybody to offer services at any scale

SaaS: Runs on the cloud

Scalability sounds exciting…

- Everybody has access to virtually unlimited resources, if needed
- No need to invest up-front

…BUT:

You need to design scalability into the Software

- Software architecture must support scalability (and any other cloud feature)



"*The ~~network~~ browser is the computer*"

Sun microsystems, 1980s

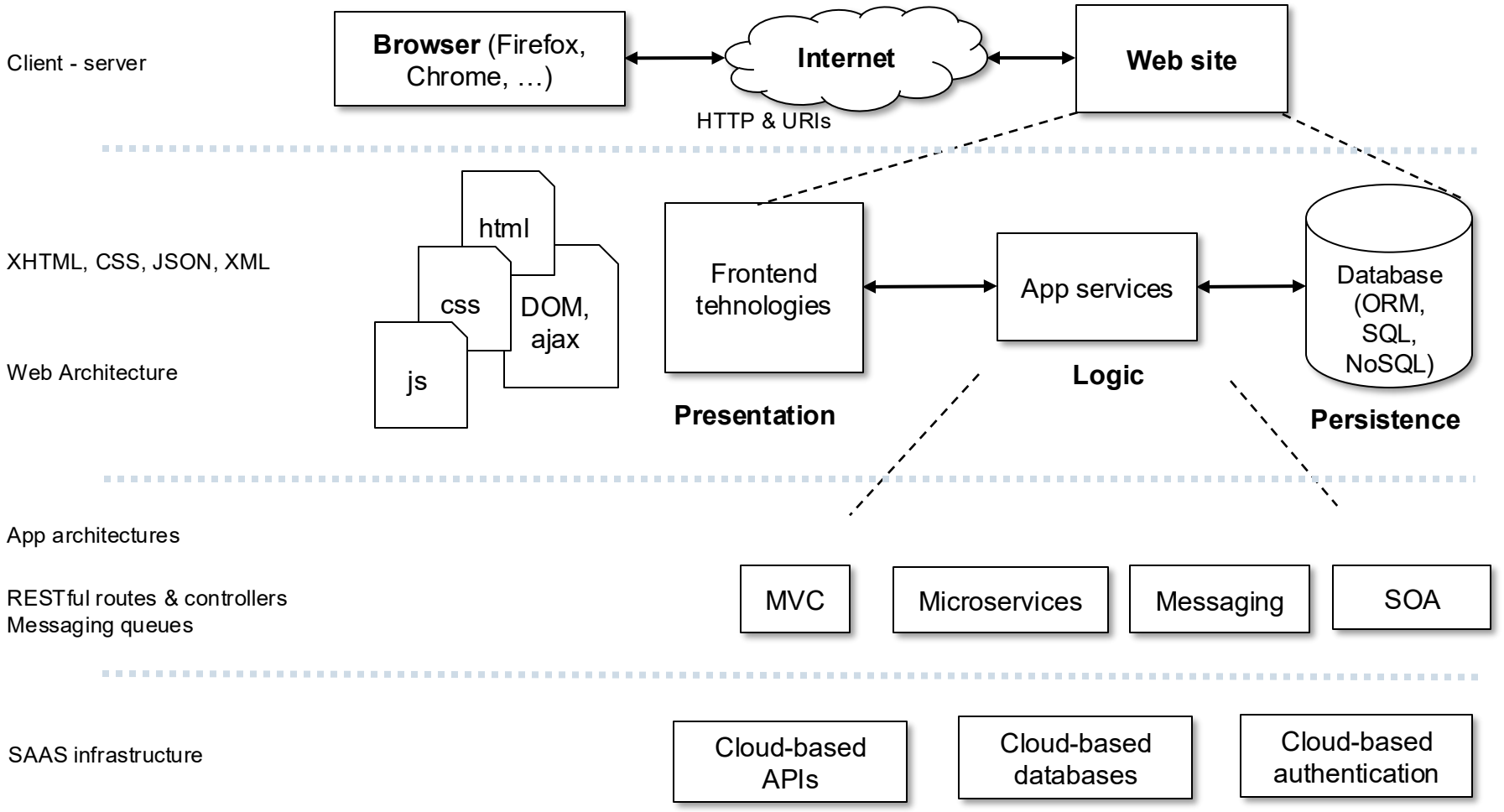
Registered slogan (2019!), by Cloudflare 

Deploying SaaS: browsers + mobile

*The **browser** is the computer!*

- With SaaS, the web becomes more "browser based"
- The browser becomes a "universal client" and its technologies become more important
  - **HTML, DOM**: Hierarchical collection of elements, for describing and managing a document's logical structure
  - **CSS**: Describe visual attributes of elements
  - **JS + Frameworks**: Client-side intelligence
- Native languages really needed only for apps that *have to* run on specific platforms

SaaS course reference technology stack



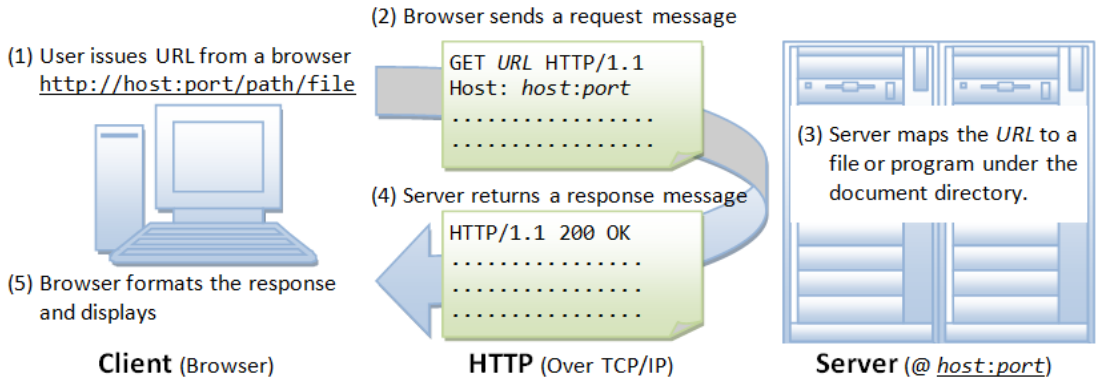
Technology stack: back-end vs. front-end



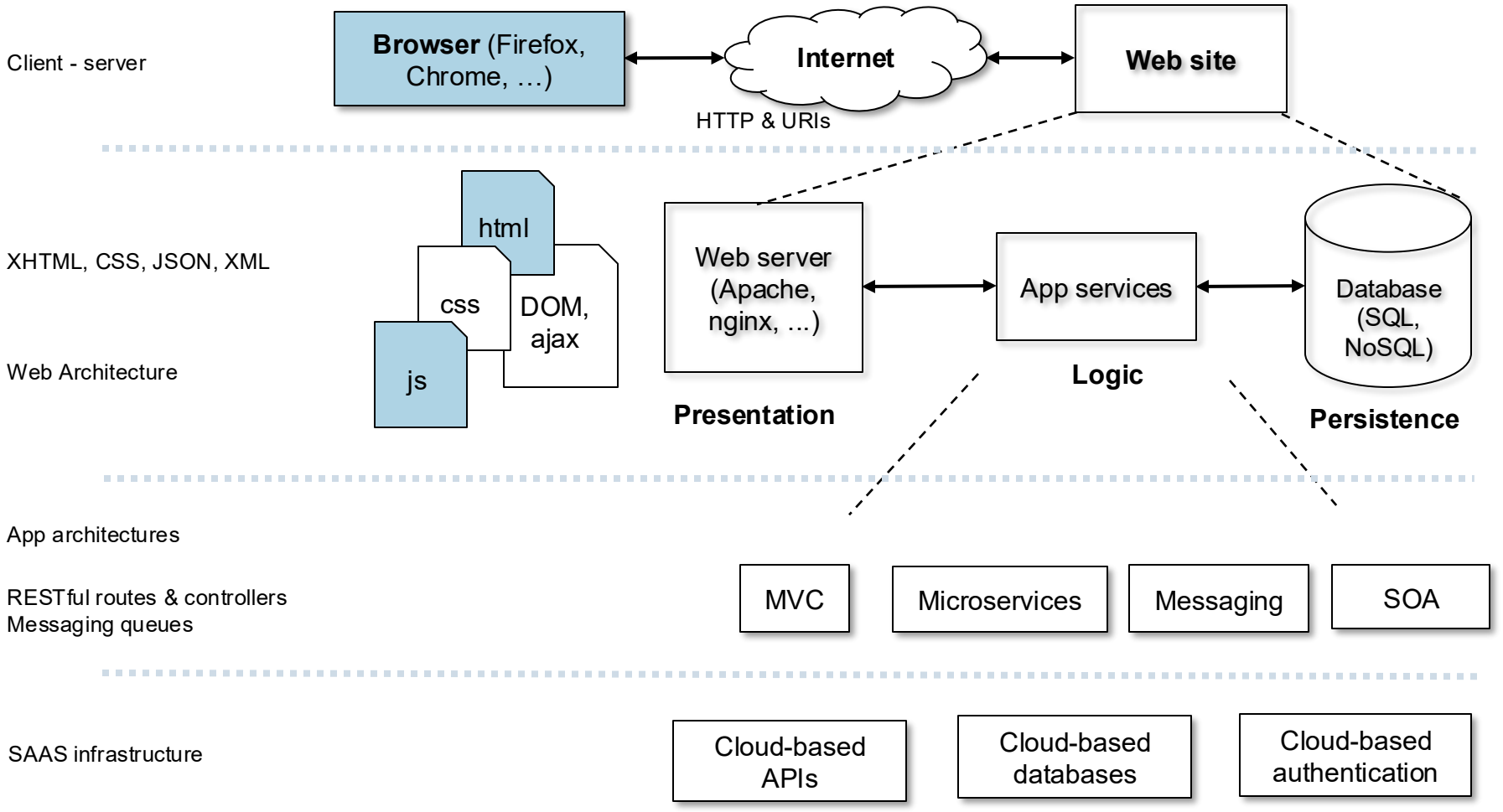
Frontend & Backend: HTTP

HTTP requests – responses

- Clients send requests to servers (pull)
- Server send back responses
- A mechanism of request methods, error codes and headers allows HTTP to be used for more than "hypertext"



Client-side scripting



Client-side scripting

How client-side scripting works

- Response received by the server contains a recognizable script
- The browser decides where to execute it
- The output of the script is rendered as any other content (HTML is still the host)

- Security warning:
  scripts actually run on the client machine (yours!)

HTTP request methods

GET	The GET method is used to retrieve information from the given server using a given URI. Requests using GET should only retrieve data and should have no other effect on the data.
HEAD	Same as GET, but it transfers the status line and the header section only.
POST	A POST request is used to send data to the server, for example, customer information, file upload, etc. using HTML forms.
PUT	Replaces all the current representations of the target resource with the uploaded content. Multiple calls do not create multiple new resources (*).
DELETE	Removes all the current representations of the target resource given by URI.
CONNECT	Establishes a tunnel to the server identified by a given URI.
OPTIONS	Describe the communication options for the target resource.
TRACE	Performs a message loop back test along with the path to the target resource.

GET vs POST

	GET	POST
BACK button/Reload	Harmless	Data will be re-submitted (alert by the browser)
Bookmarked	Can be bookmarked	Cannot be bookmarked
Cached	Can be cached	Not cached
Encoding type	application/x-www-form-urlencoded	application/x-www-form-urlencoded or multipart/ form-data. Use multipart encoding for binary data
History	Parameters remain in browser history	Parameters are not saved in browser history
Restrictions on data length	Yes, when sending data, the GET method adds the data to the URL; and the length of a URL is limited (maximum URL length is 2048 characters)	No restrictions
Restrictions on data type	Only ASCII characters allowed	No restrictions
Security	Less secure than POST because data sent is part of the URL. Not to be used for sending passwords!	POST is a little safer than GET because the parameters are not stored in browser history or in web server logs
Visibility	Data is visible to everyone in the URL	Data is not displayed in the URL

POST can be used anywhere instead of GET; It is disputable, however, if this is a good practice

Frontend: The Document Object Model

HTML: a complete description of a web site

- Structure (e.g. header, body, divs)
- Static content, created by an editor
- Dynamic content, created at load/runtime
  either on the server or the client (browser)

DOM: a representation of an HTML document

- Created by the browser
- Allows the elements of a web page
  to be accessed as objects

Access to DOM nodes

Selection & querying

```
let button3 = document.getElementById('bordersButton');


button3.addEventListener('click', () => {
    const level3 = document.querySelectorAll('h3');
    level3.forEach(item => {
        if (item.style.border === '1px solid red')
            item.style.border = 'none';
        else
            item.style.border = '1px solid red';
    });
```

Working with the DOM in js: jQuery

jQuery: what is it

- Javascript library for DOM manipulation

jQuery: why need it

- Browser incompatibilities in DOM management

JavaScript

```
document.getElementById("myHello").innerHTML = "Hello SaaS World!";
```

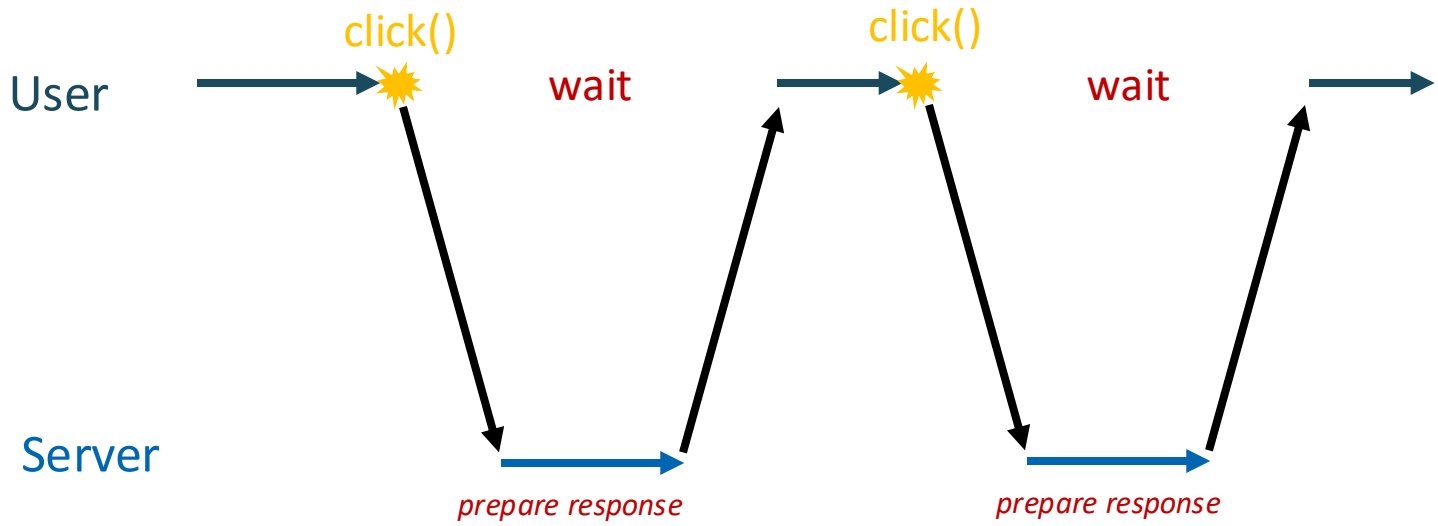
jQuery

```
$("#myHello").html("Hello SaaS World!");
```

Ajax: **A**synchronous **J**avasctipt **A**nd **X**ML

Synchronous web communication

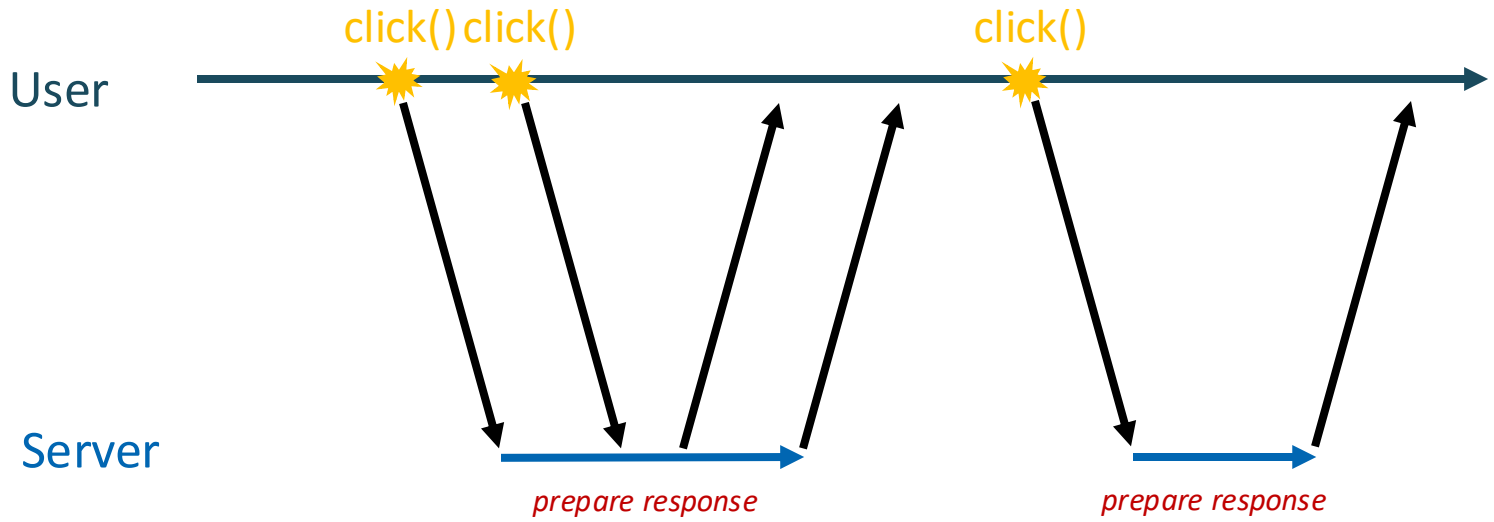
- Click ->
- Wait ->
- Refresh page



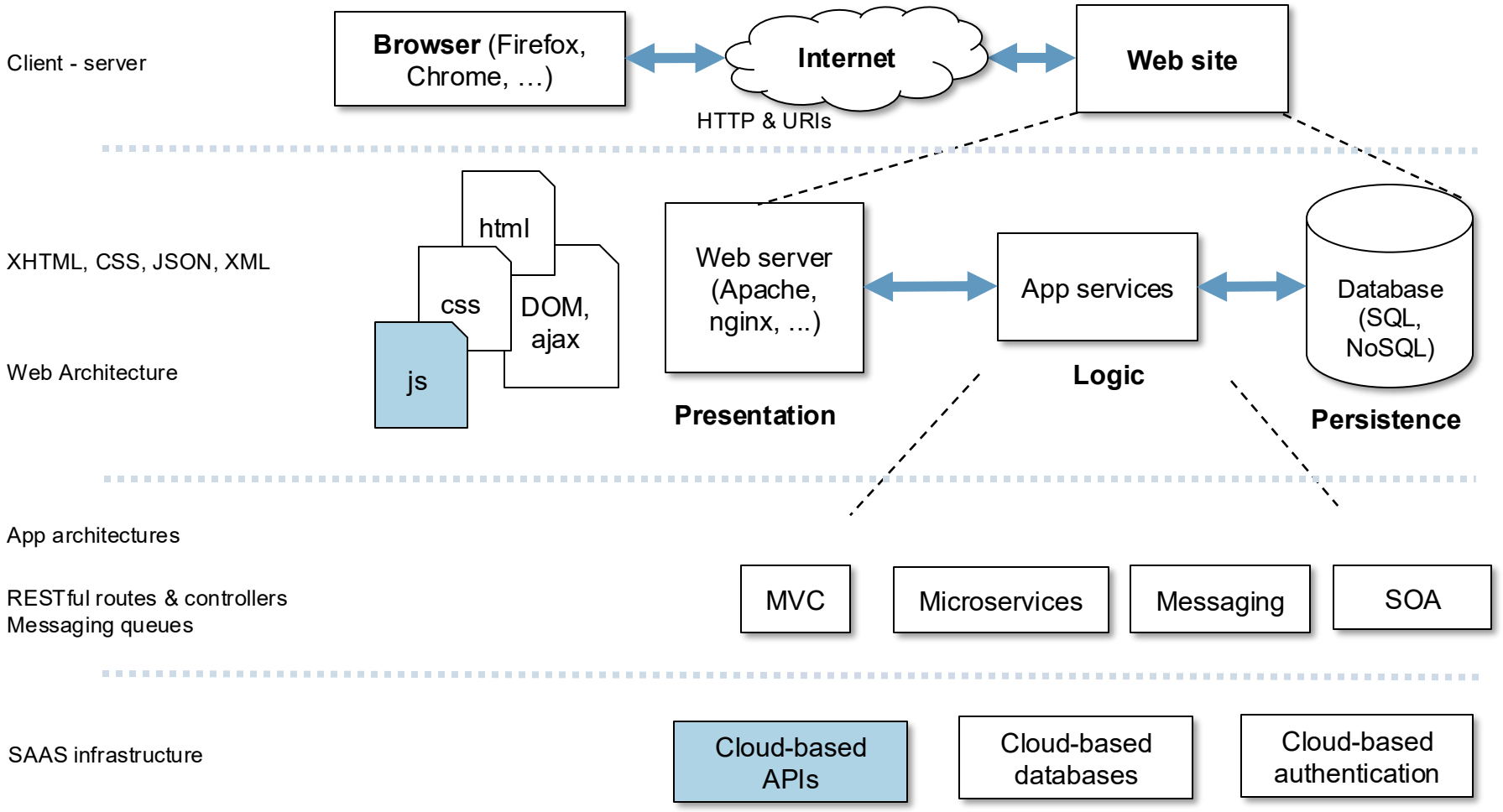
Ajax: **A**synchronous **J**avasctipt **A**nd **X**ML

Asynchronous web communication

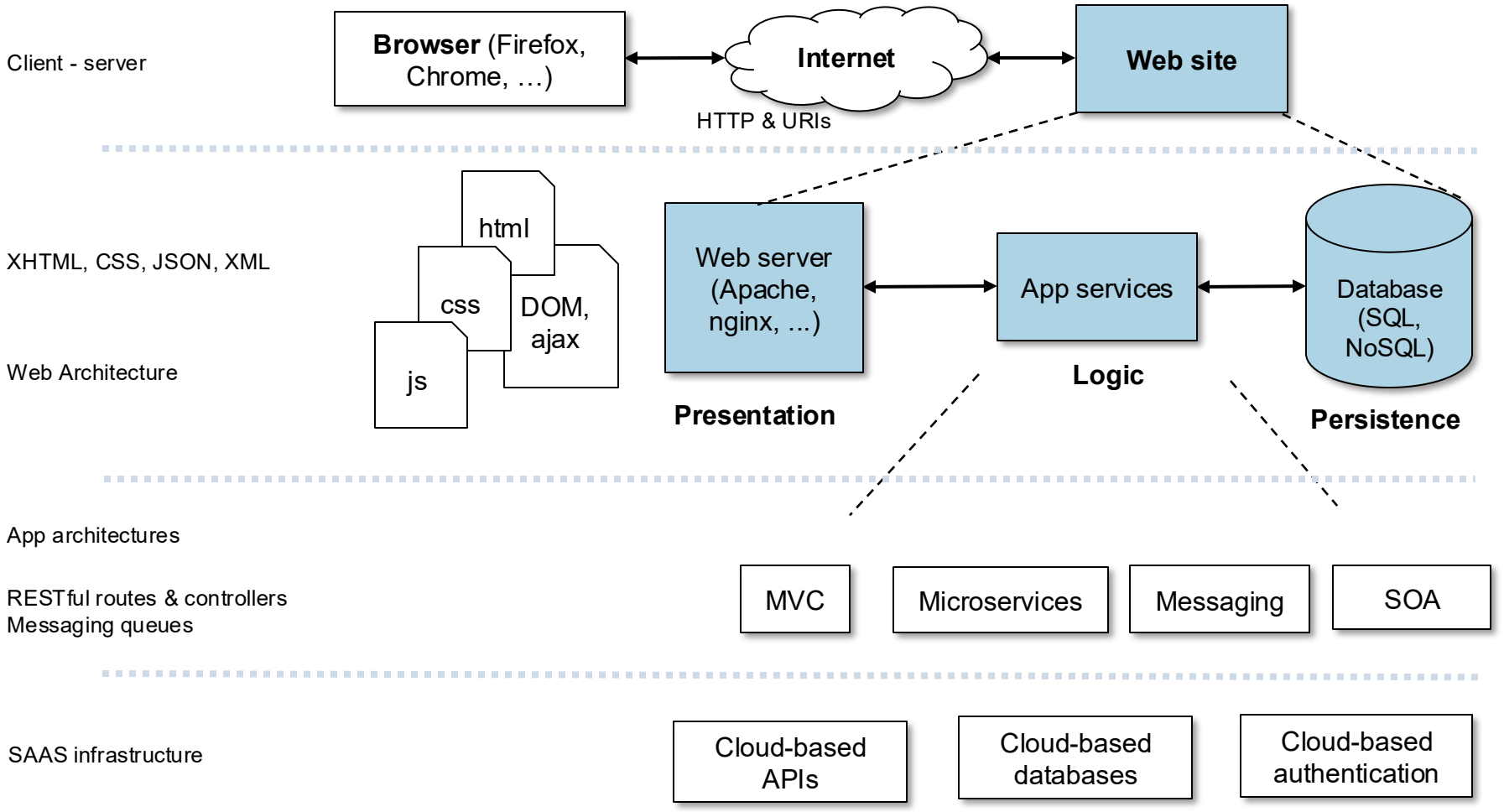
- Click ->
- Continue working ->
- Refresh affected page element



JSON for data exchange



Backend: Server-side scripting

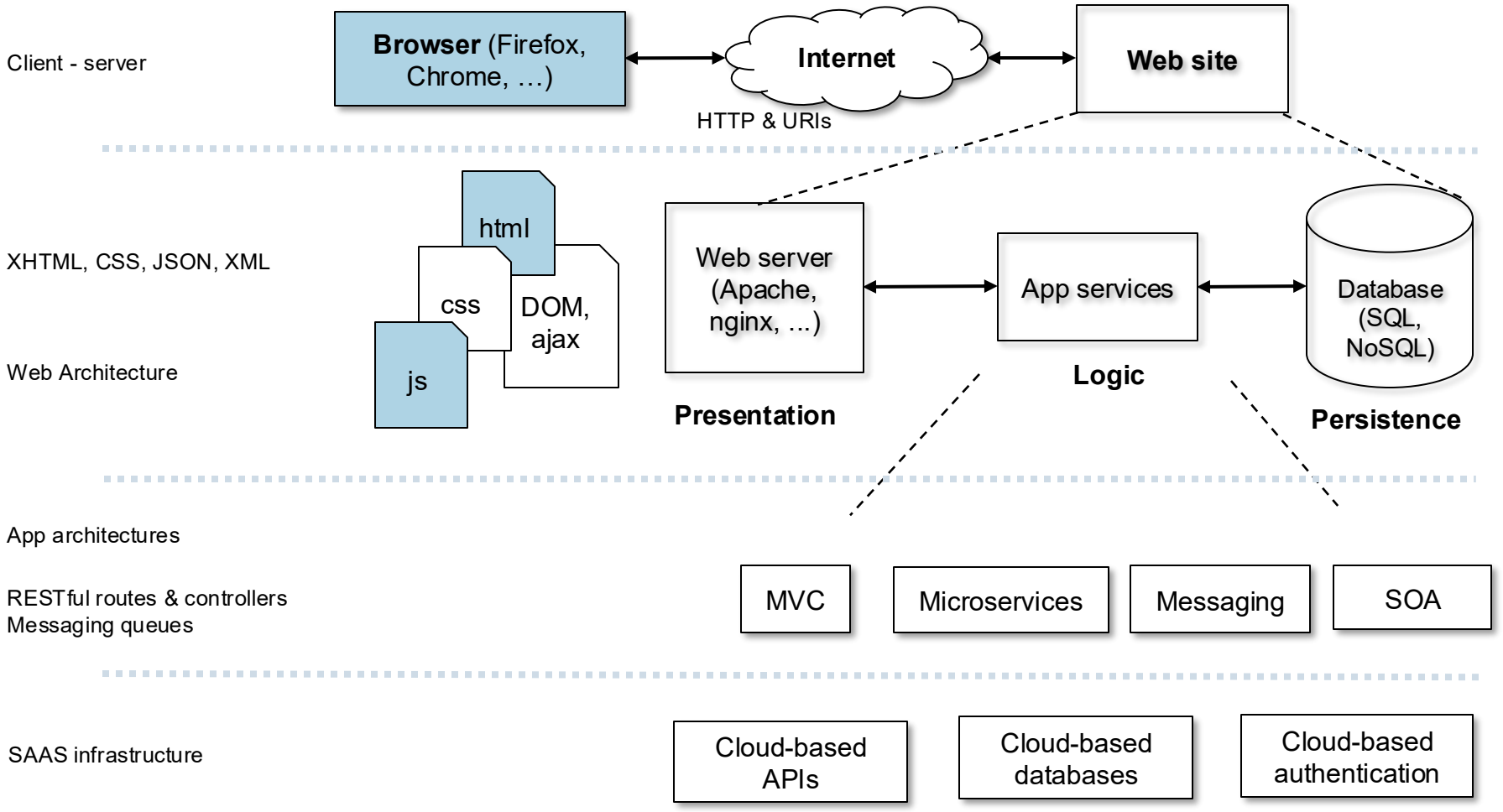


Server-side scripting

How server-side scripting works

- Request received by the client
- Server runs locally a program (script)
- The output of the program is HTML or any other text
- The resulting text is sent to the client using HTTP
- The client does not know if the received data comes from a script or is static text
- Scripts have local privileges on the server as needed for connecting to databases or other services

Client-side scripting



Client-side scripting may also be used for jobs that would also fit in the backend: Be aware of what you are doing!

Client-side scripting

How client-side scripting works

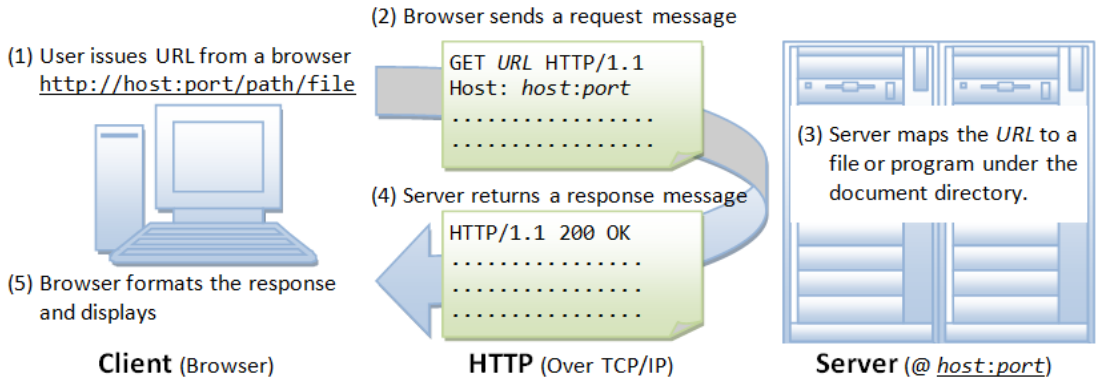
- Response received by the server contains a recognizable script
- The browser decides where to execute it
- The output of the script is rendered as any other content (HTML is still the host)

- Security warning:
  scripts actually run on the client machine (yours!)

Frontend & Backend: HTTP

HTTP requests – responses

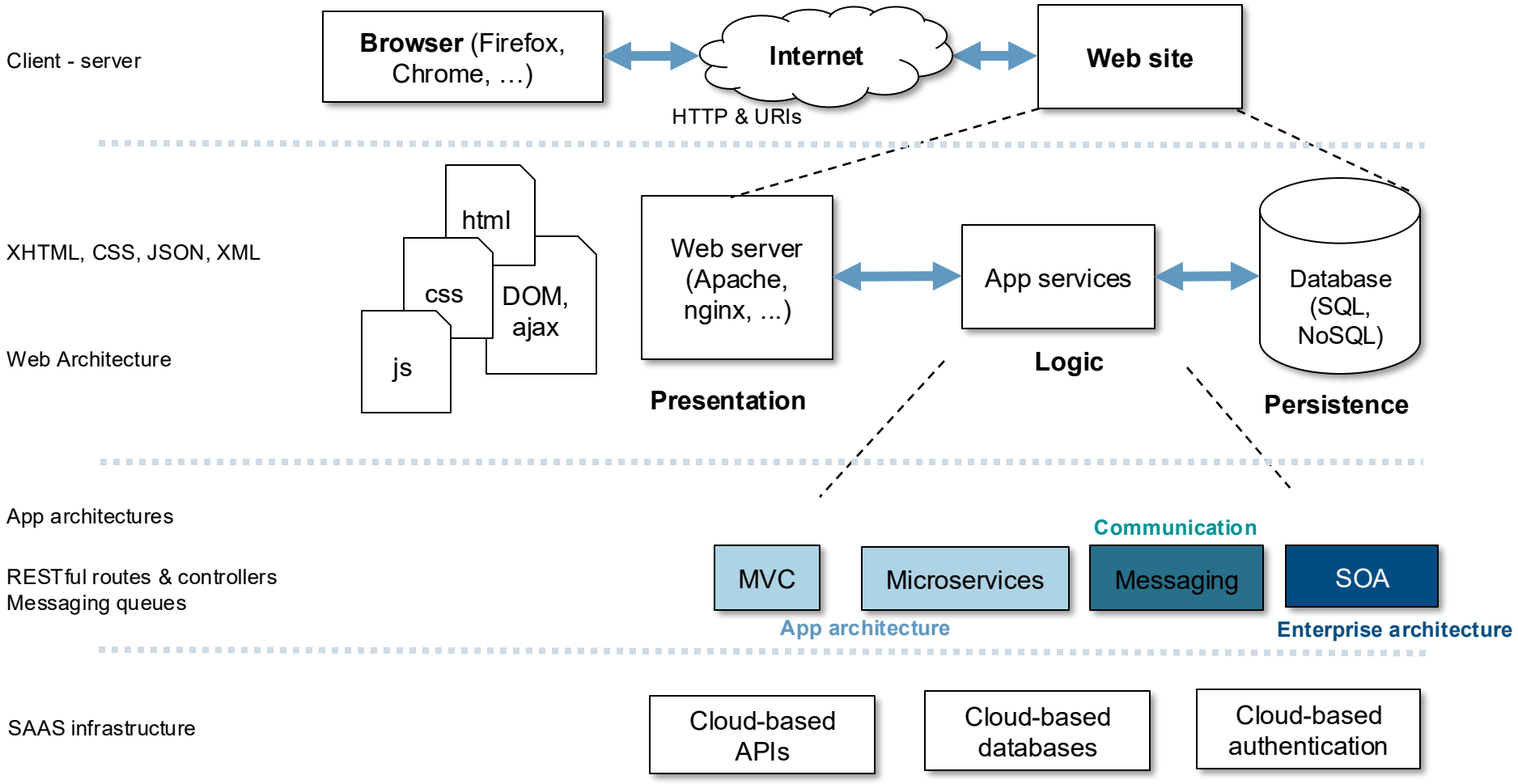
- Clients send requests to servers (pull)
- Server send back responses
- A mechanism of request methods, error codes and headers allows HTTP to be used for more than "hypertext"



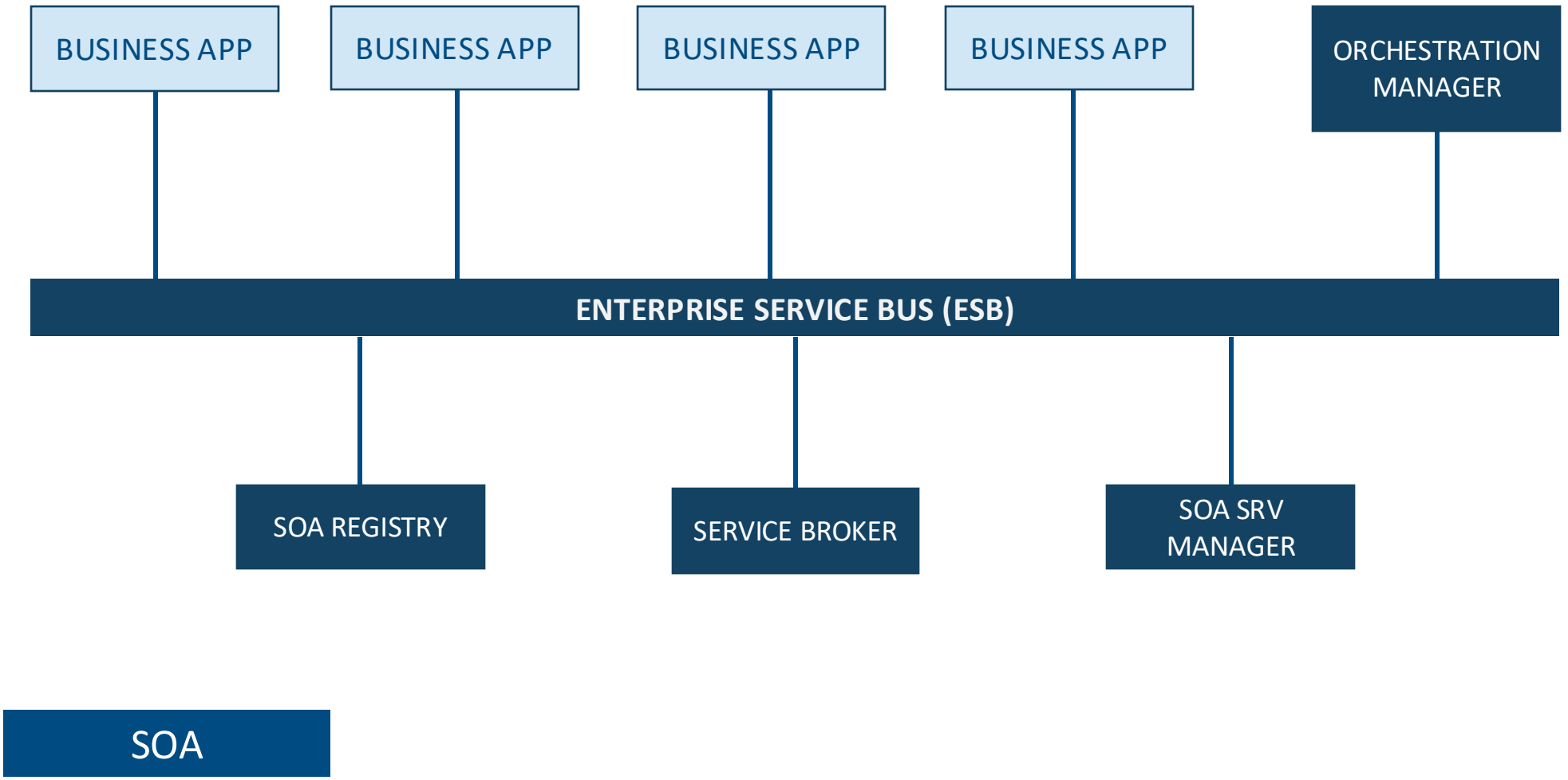
HTTP request methods

GET	The GET method is used to retrieve information from the given server using a given URI. Requests using GET should only retrieve data and should have no other effect on the data.
HEAD	Same as GET, but it transfers the status line and the header section only.
POST	A POST request is used to send data to the server, for example, customer information, file upload, etc. using HTML forms.
PUT	Replaces all the current representations of the target resource with the uploaded content. Multiple calls do not create multiple new resources (*).
DELETE	Removes all the current representations of the target resource given by URI.
CONNECT	Establishes a tunnel to the server identified by a given URI.
OPTIONS	Describe the communication options for the target resource.
TRACE	Performs a message loop back test along with the path to the target resource.

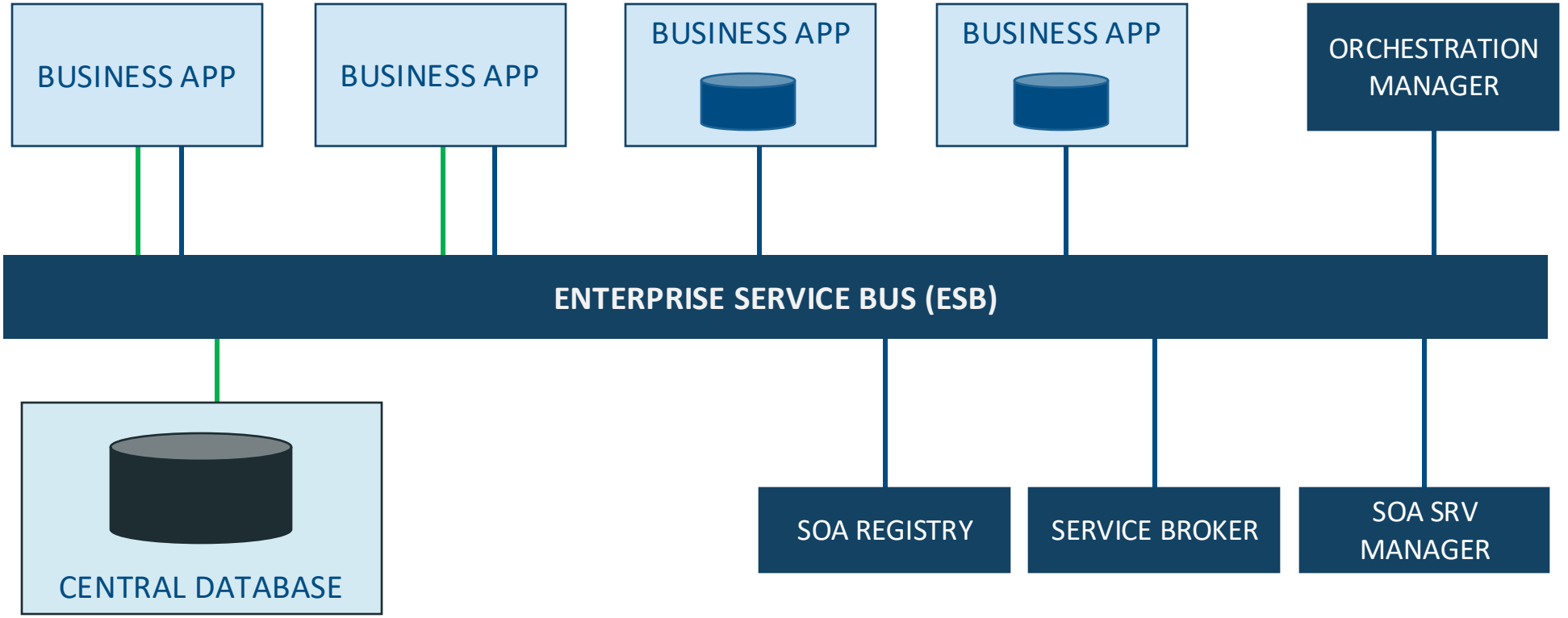
Architectures



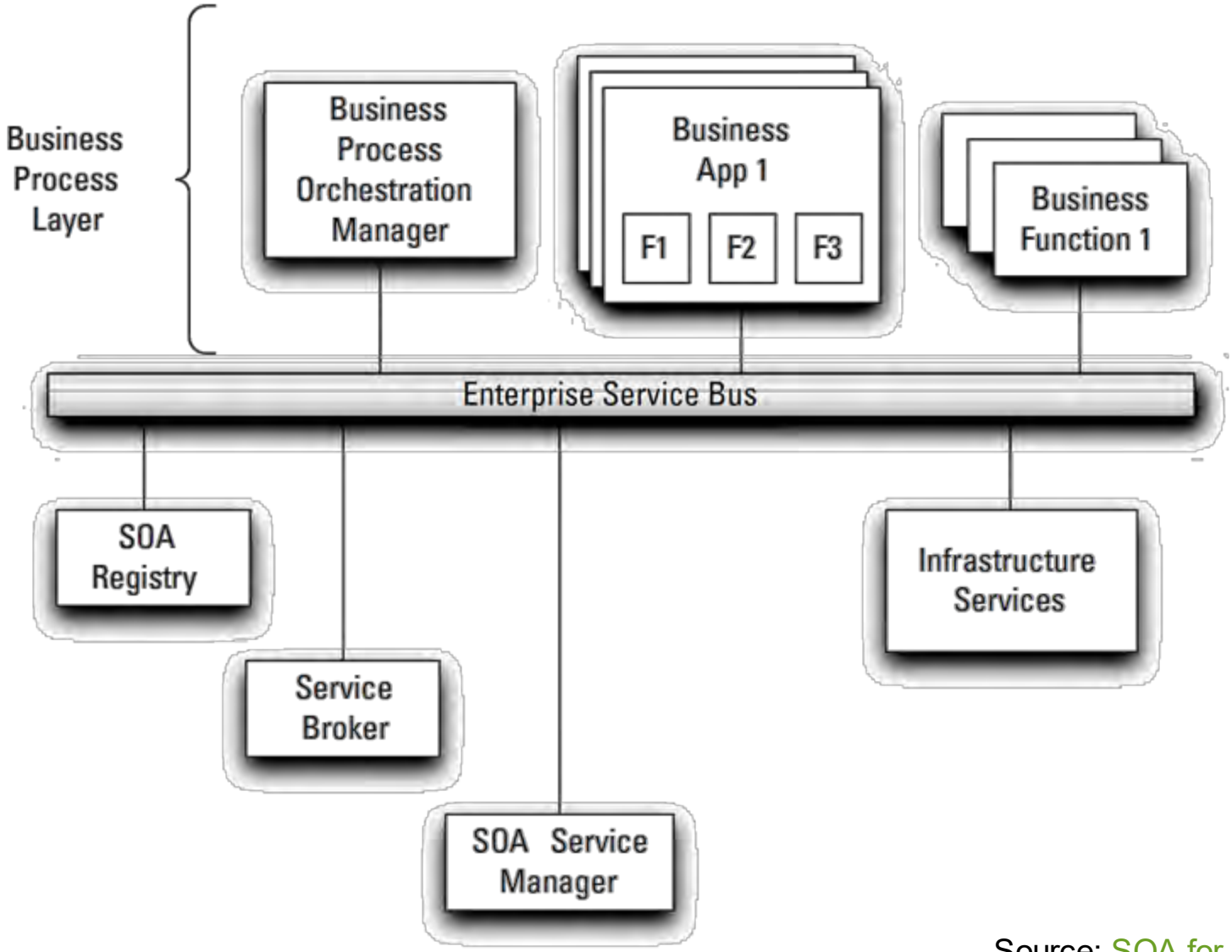
Service-Oriented Architecure



Service-Oriented Architecure

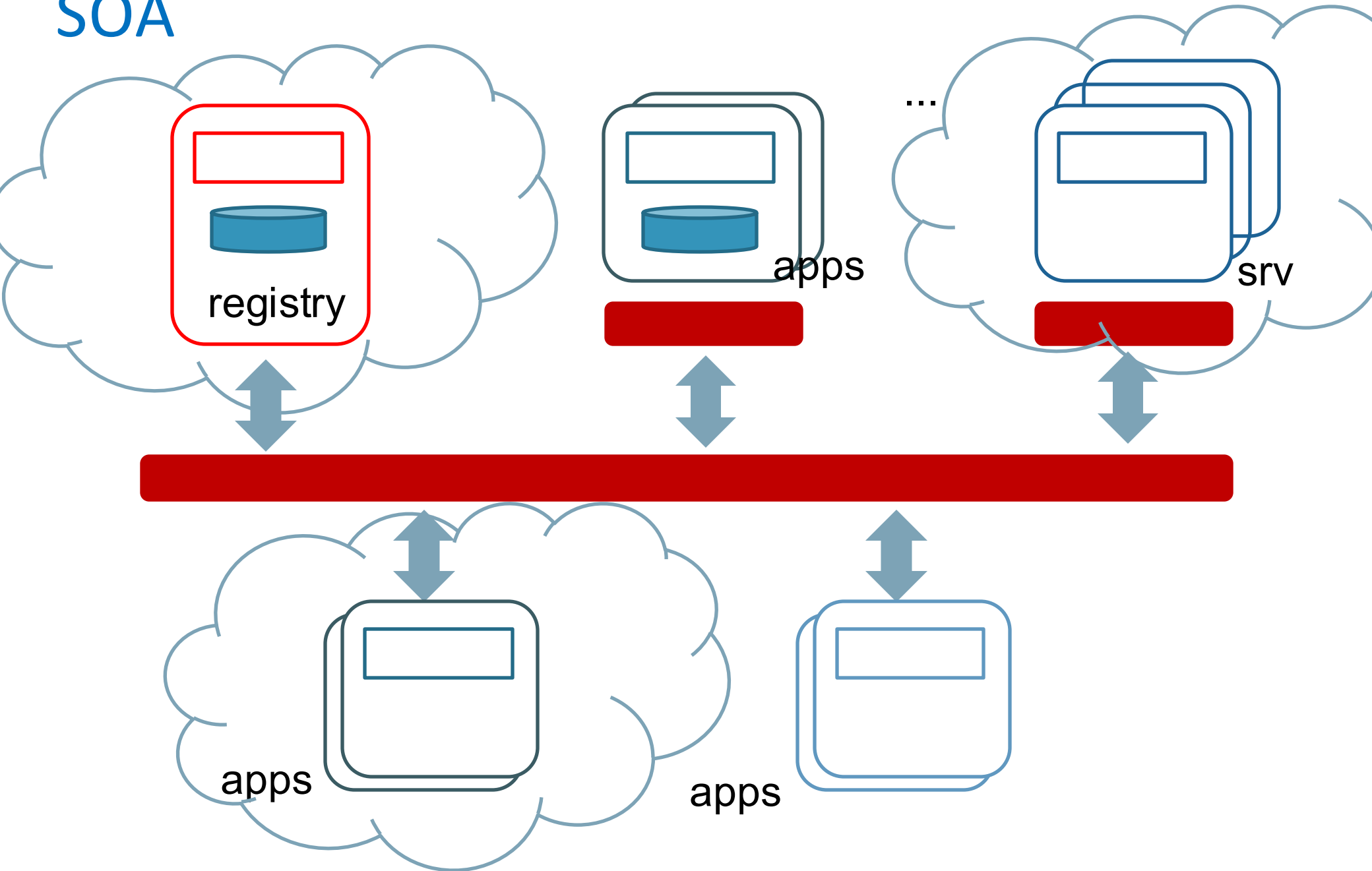


Service-Oriented Architecure

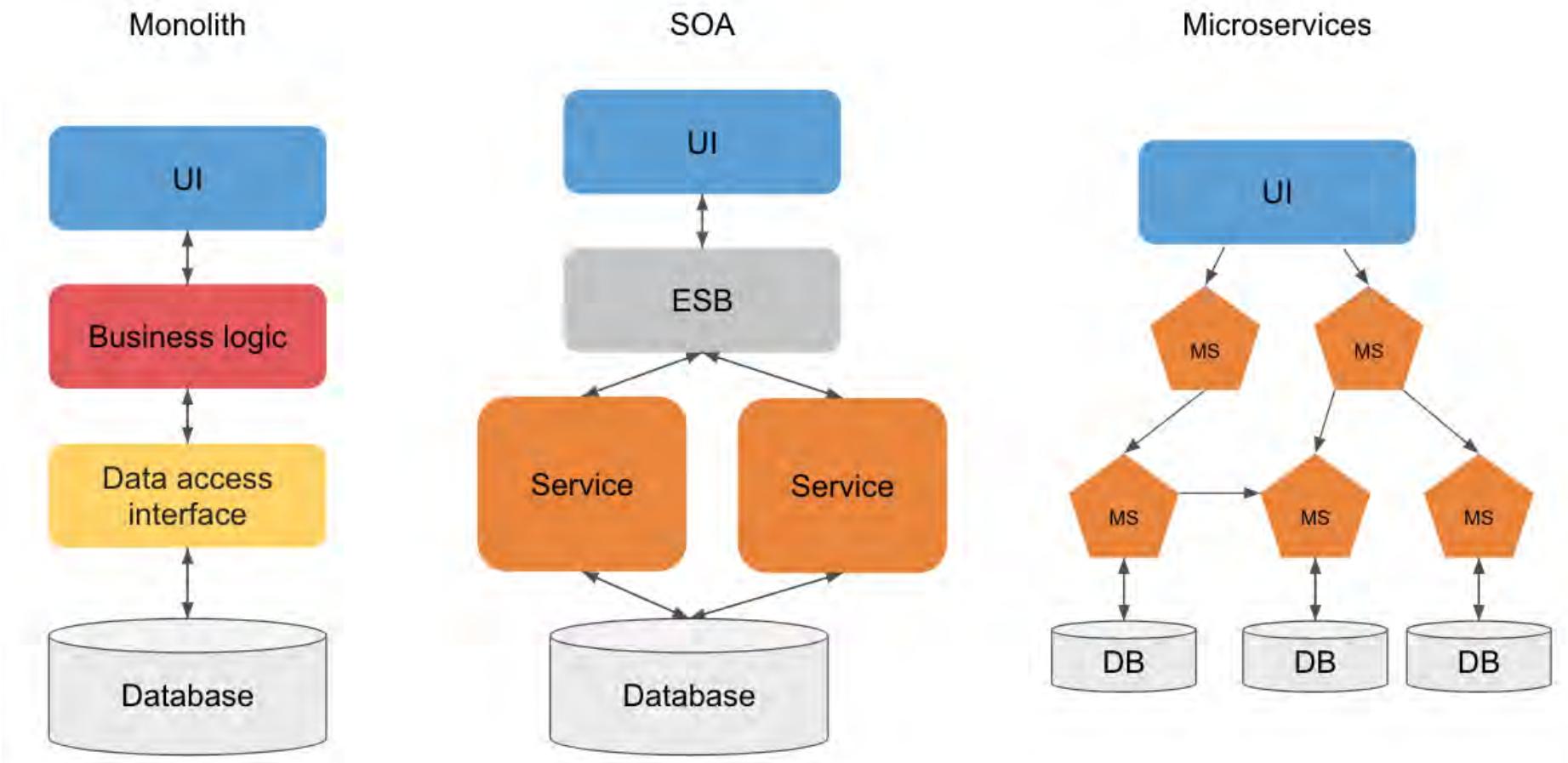


Source: SOA for dummies, free book

SOA

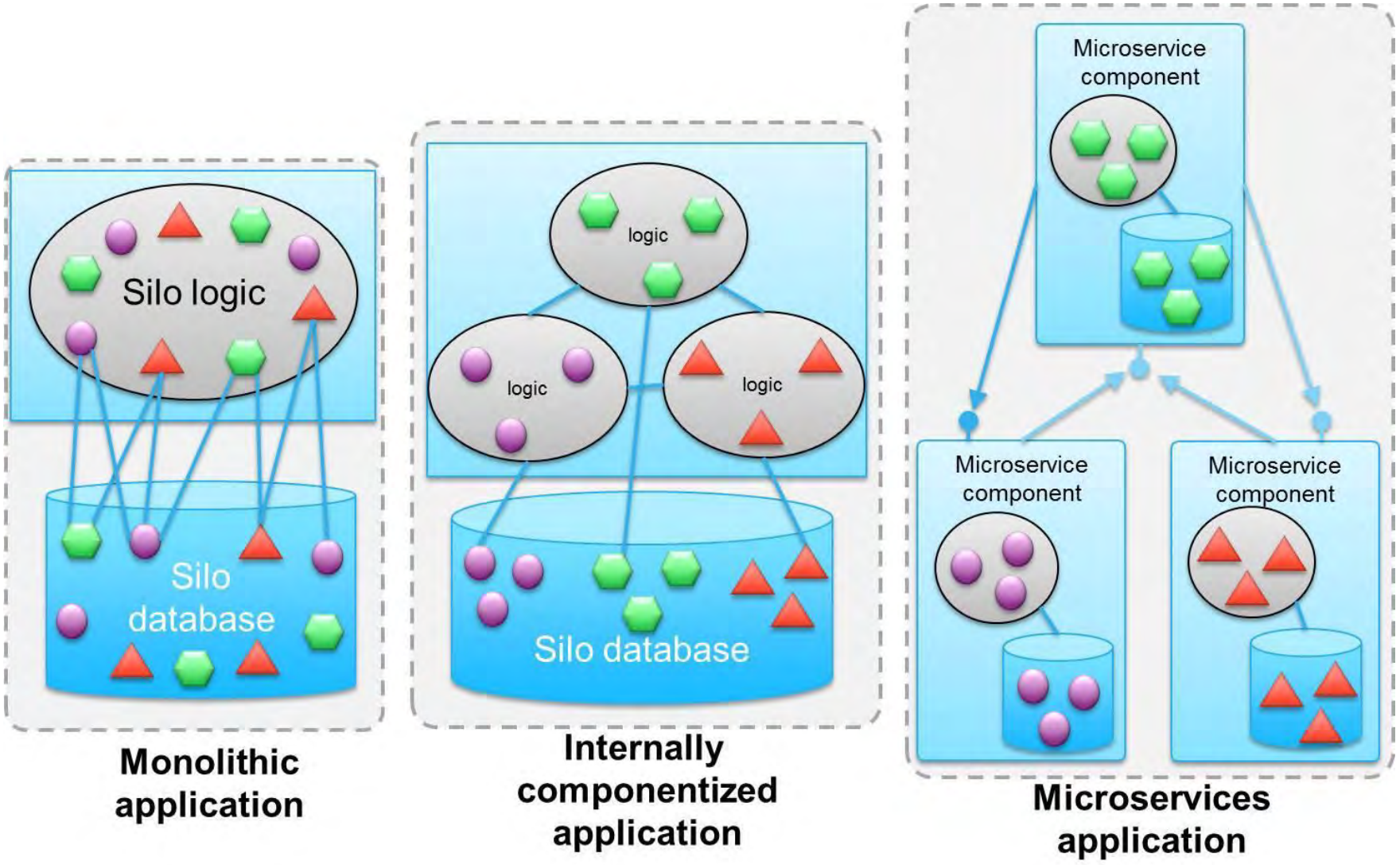


Monoliths, SOA & microservices illustrated



Source: Microservices Architecture From A to Z, S.Bouttier @Medium

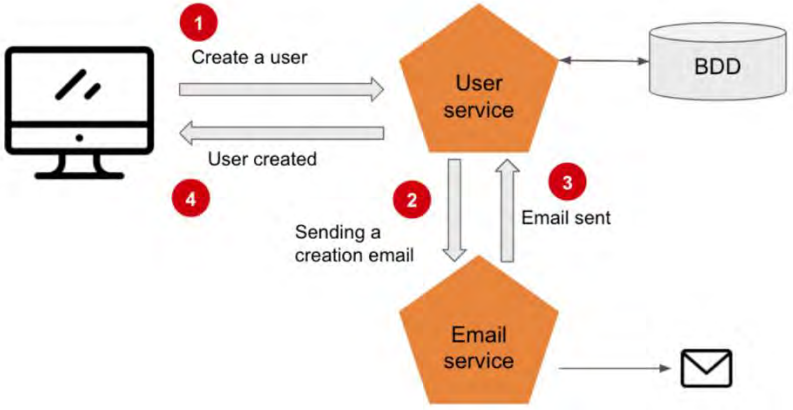
Monoliths, SOA & microservices illustrated



Source: IBM

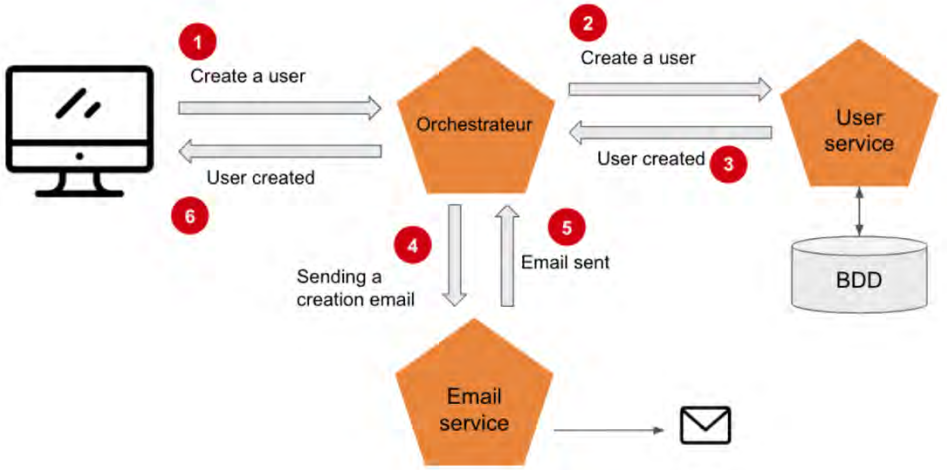
Microservices communication

Use of REST calls vs...



- Dependencies become complex
- Complex is difficult to maintain
- Behaviour is impacted
- Can do better

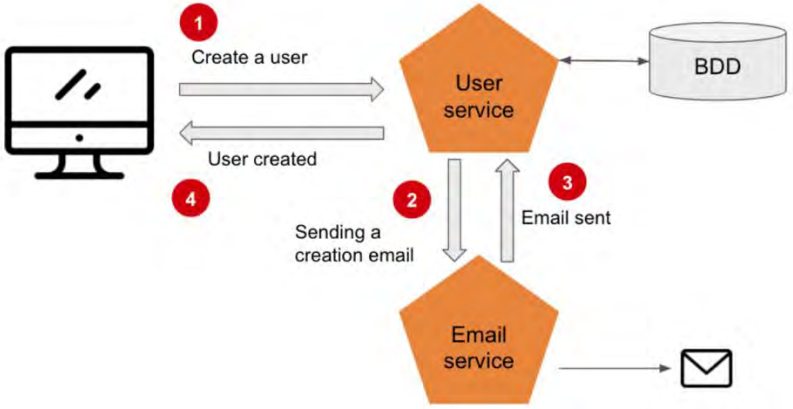
1: orchestration



- Add a service to manage dependencies ("orchestrator")
- The orchestrator should not fail

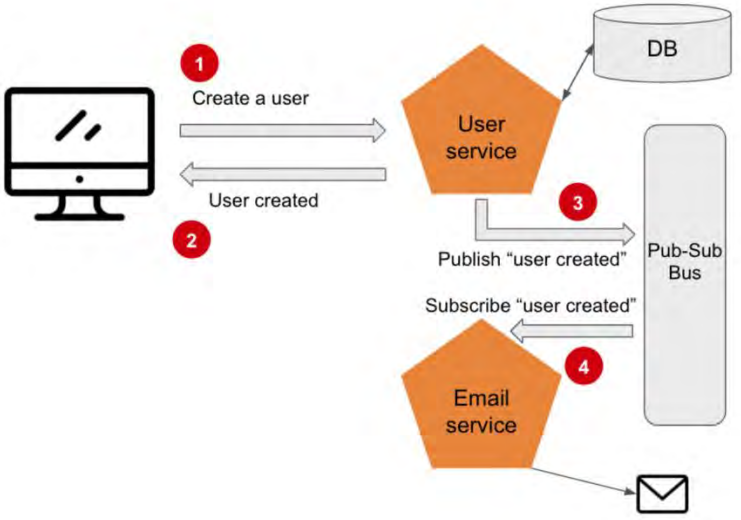
Microservices communication

Use of REST calls vs...



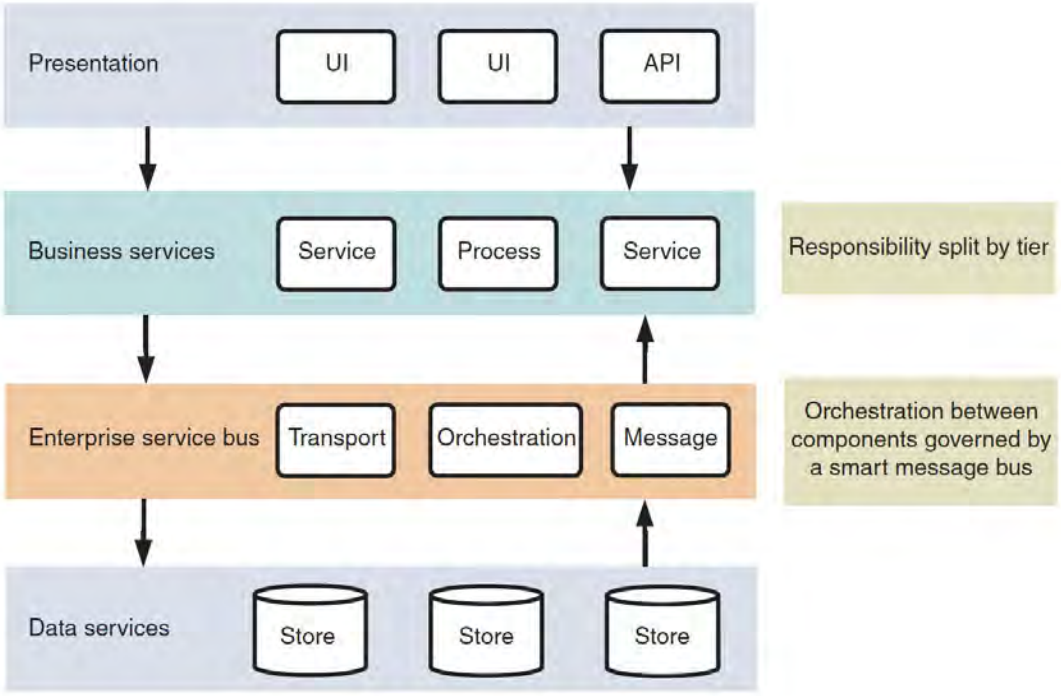
- Dependencies become complex
- Complex is difficult to maintain
- Behaviour is impacted
- Can do better

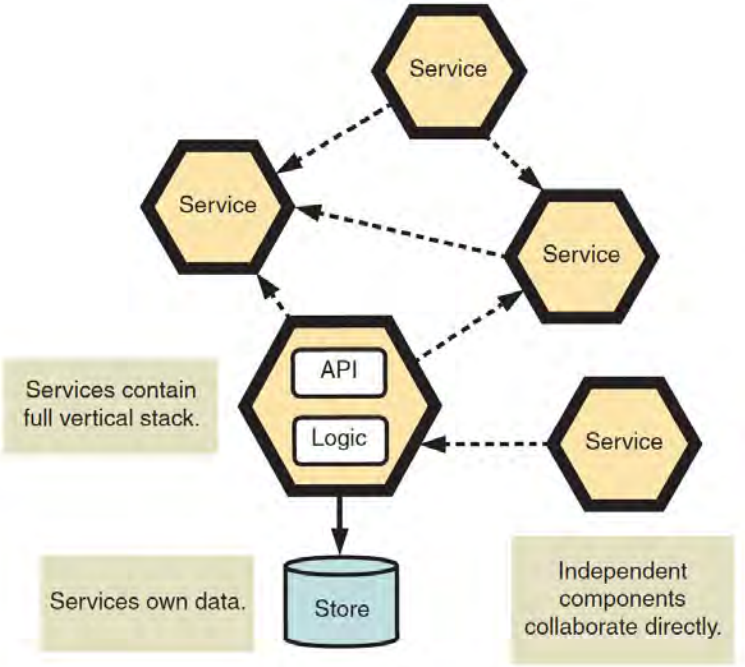
2: choreography



- Use a publish-subscribe model (no direct or mediated communication)
- Data consistency concerns

SOA & microservices illustrated

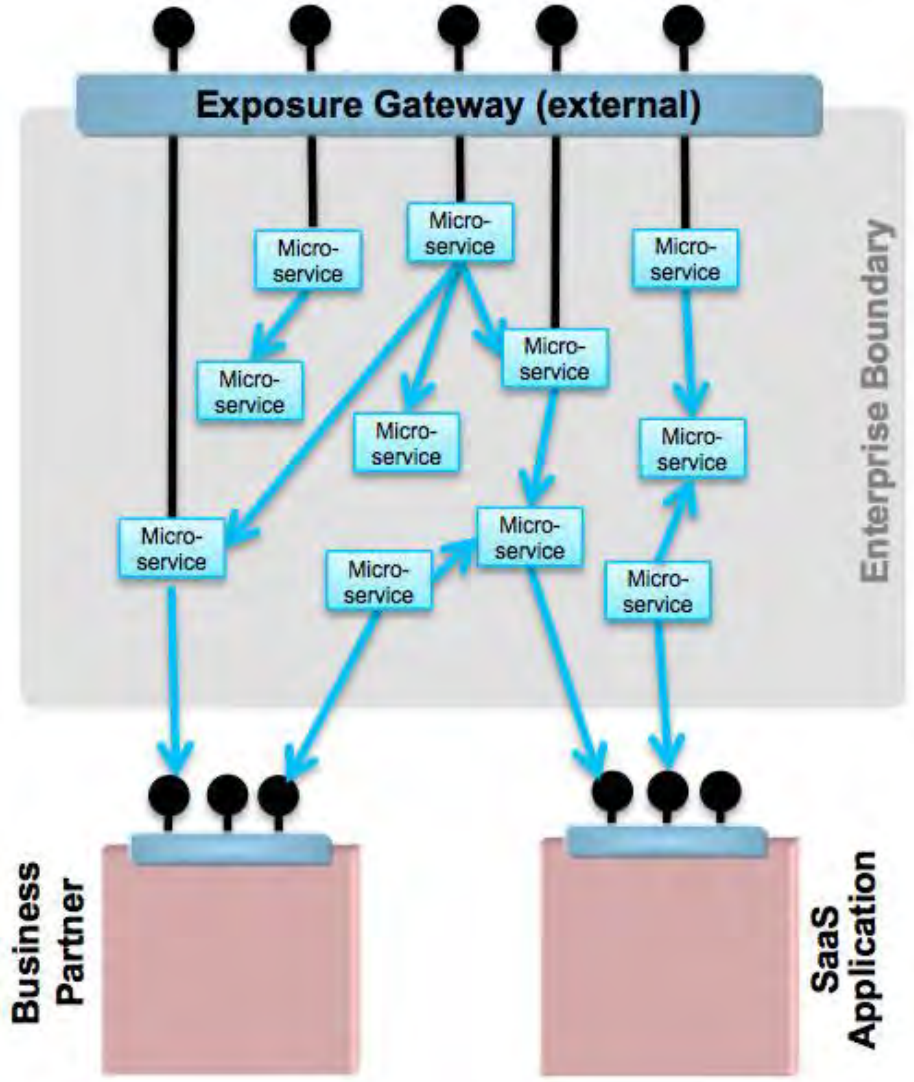




Source: Microservices in Action, M.Bruce, A.Pereira

SOA and microservices

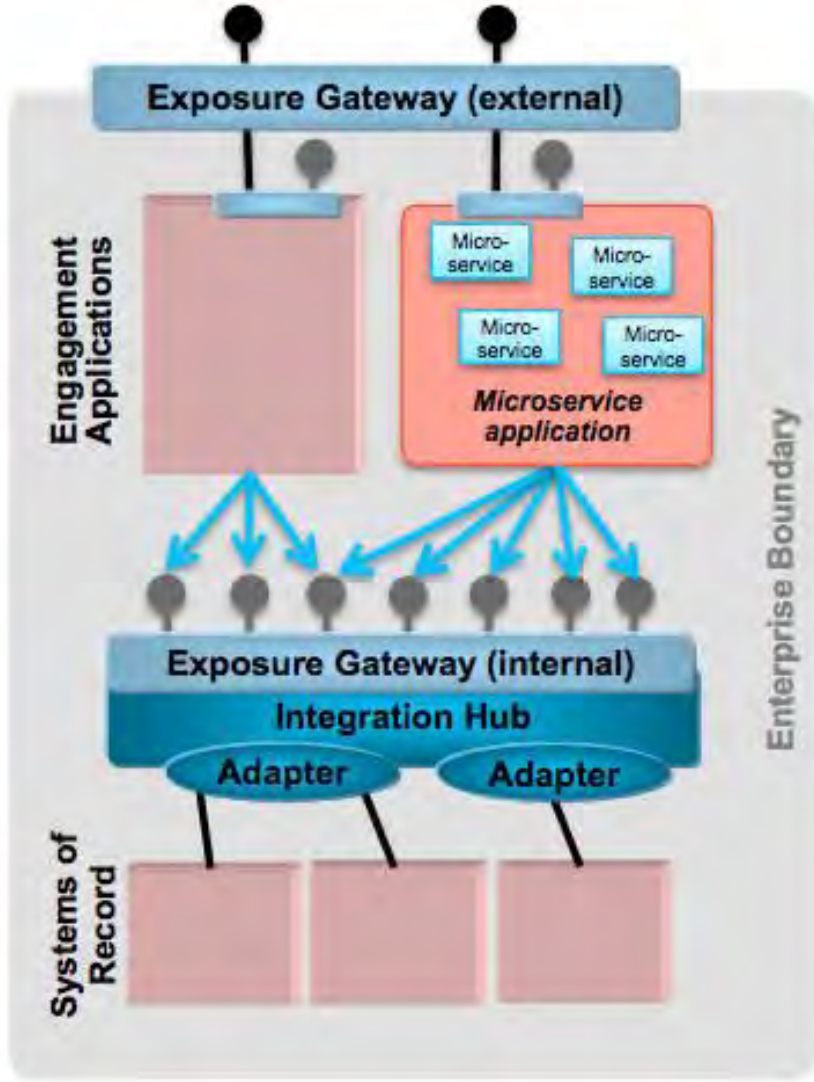
Clean development
(no legacy apps)



Source: IBM

SOA and microservices

Micoservices need to co-exist with existing apps

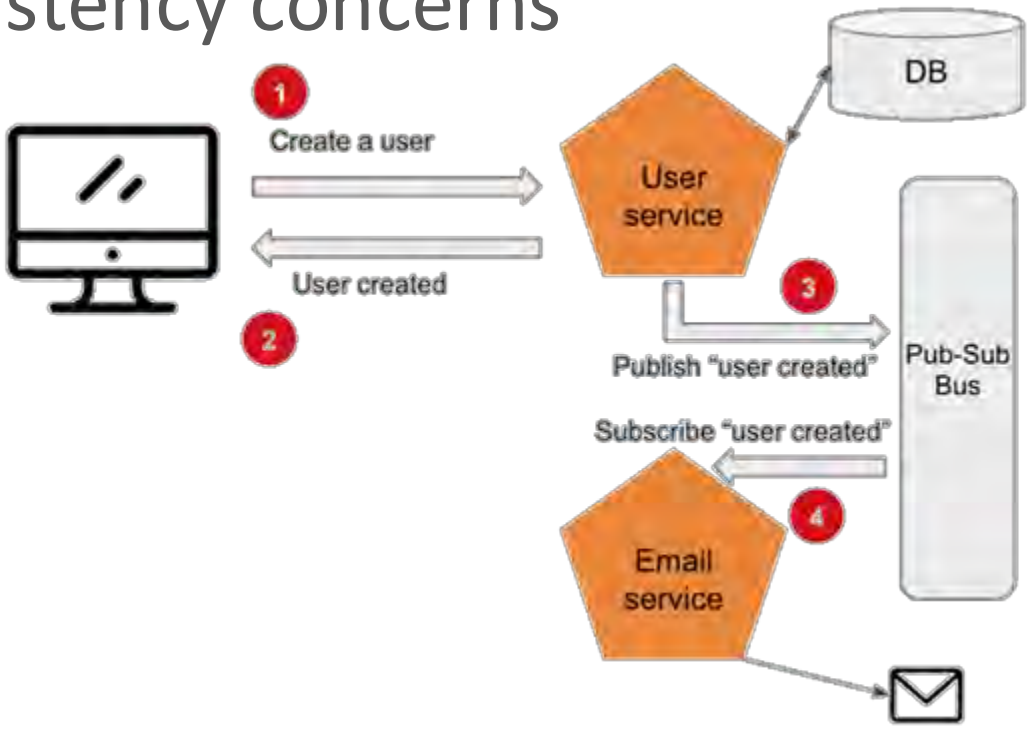


Source: IBM

Inter-process communication

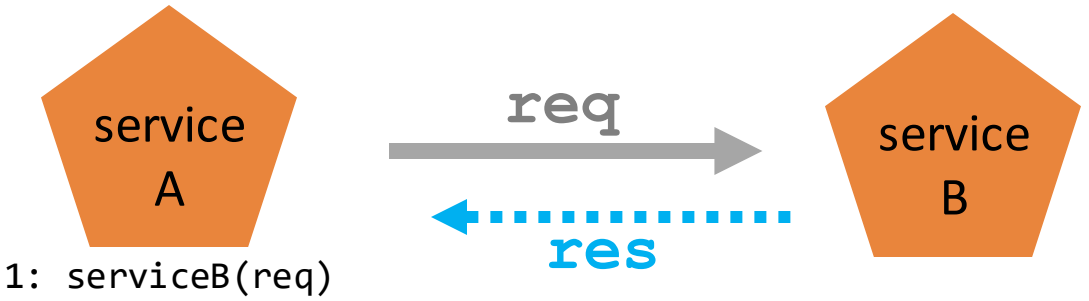
The publish-subscribe model in microservices choreography

- No direct or mediated communication
- Data consistency concerns

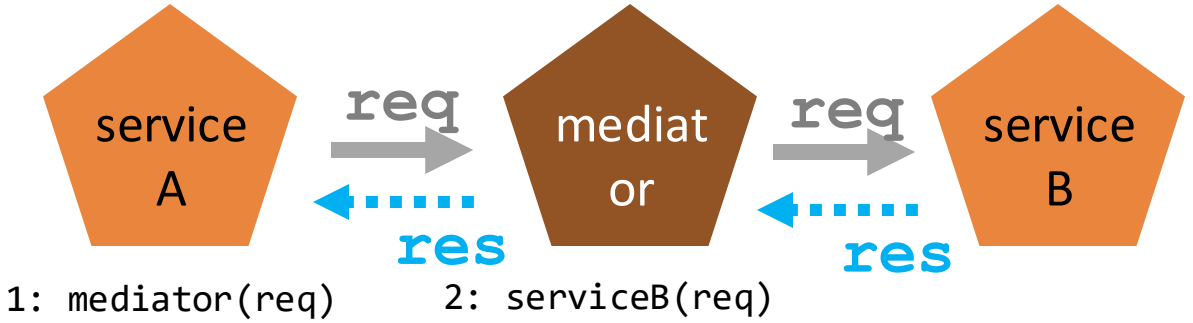


Inter-process communication

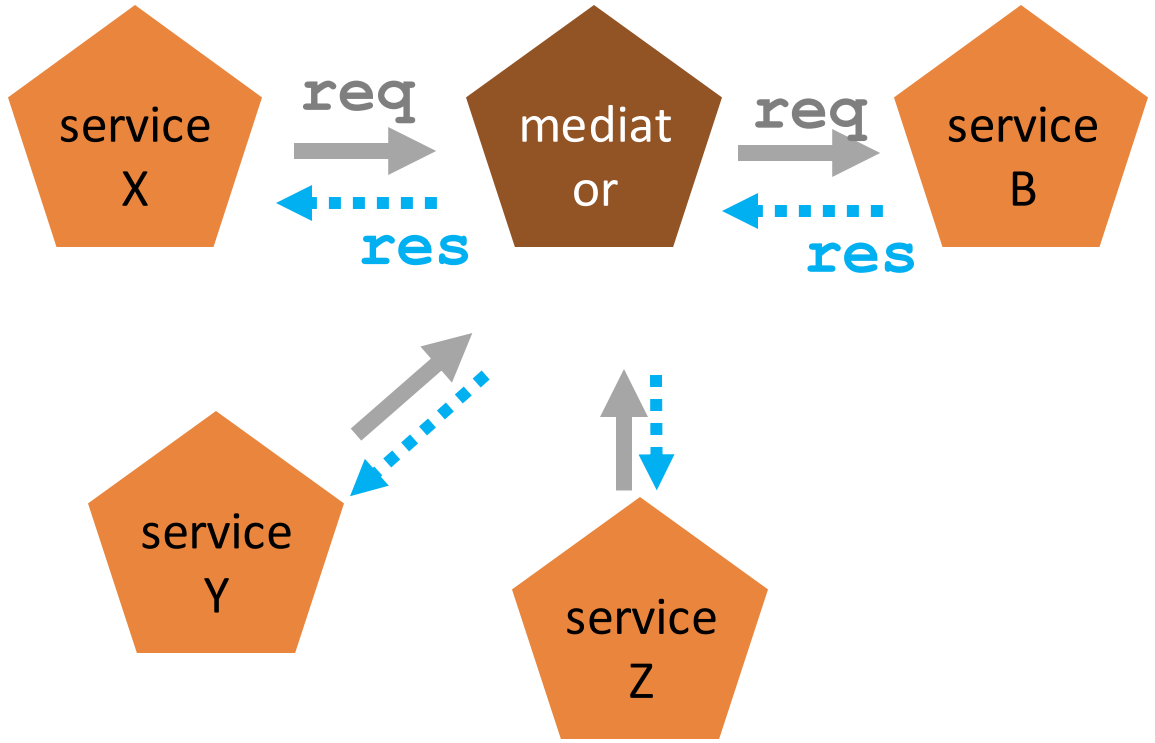
Direct communication



Mediated communication



Inter-process communication



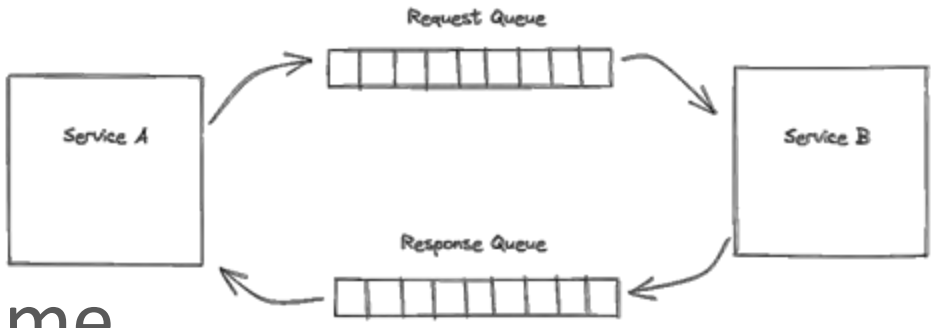
Asynchronous communication

No temporal-coupling between request, processing, response

- User performs a request and provides a callback to be notified when response is available.
- No data dependency:
  - unlock code with no dependency to the response
  - schedule code with a hard dependency to be executed when possible
- Sender availability depends only on request time

Asynchronous communication

Request time depends on



- The network
- Service dependencies' lifetime
  - An async interface on ServiceA pushes the request to a separate queue service
  - ServiceB pulls from the queue, processes requests and then pushes responses to another queue
  - ServiceA service reads from the response queue and feeds the callback.

Asynchronous communication

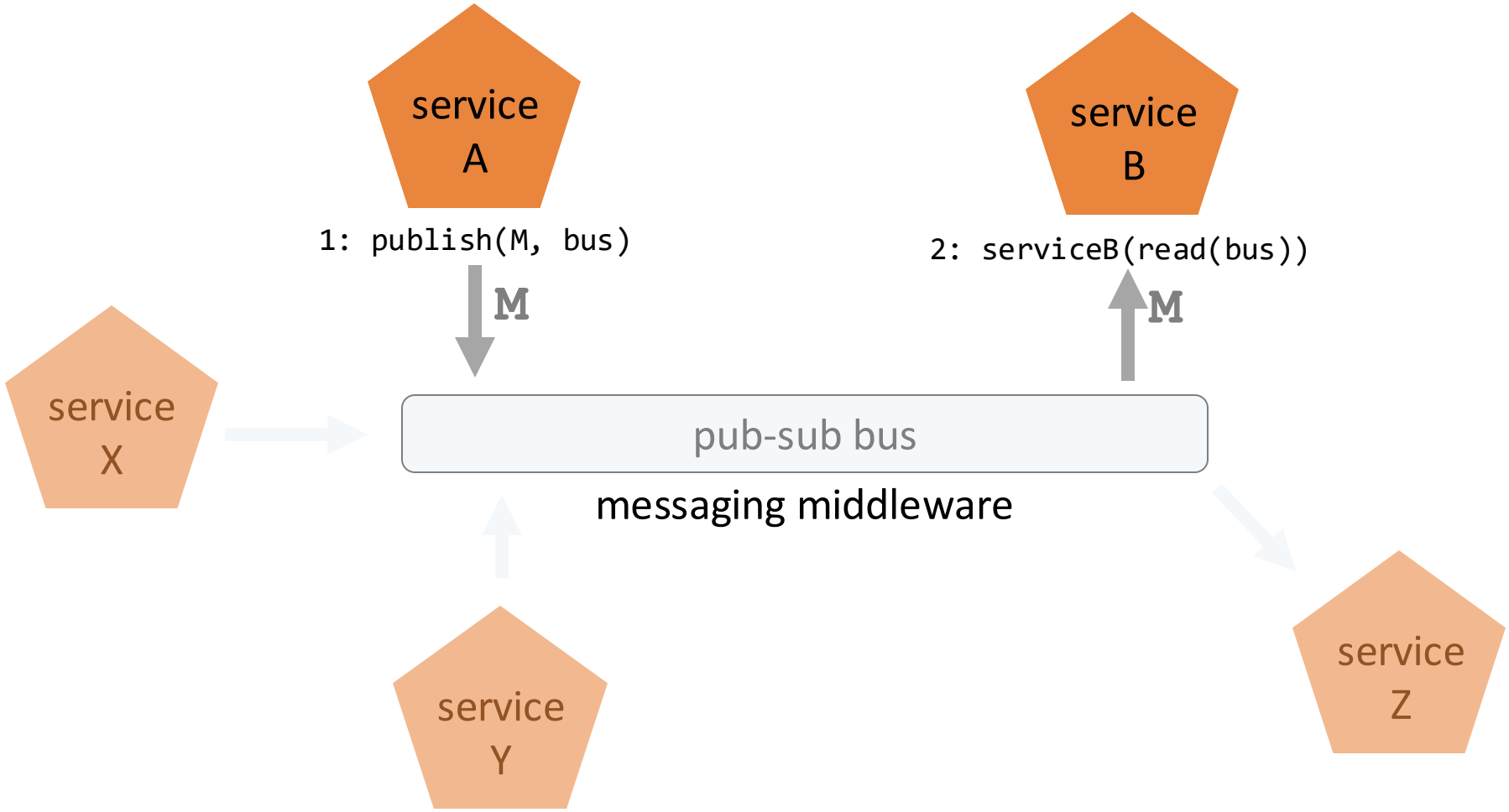
What does this mean?

- instead of ServiceB we are now coupled to the queue's lifetime: the problem is not eliminated...
- Need to provide high-availability for the queue service (single point-of-failure)

- The cost of high-availability can be shared if the queue can be used by many services

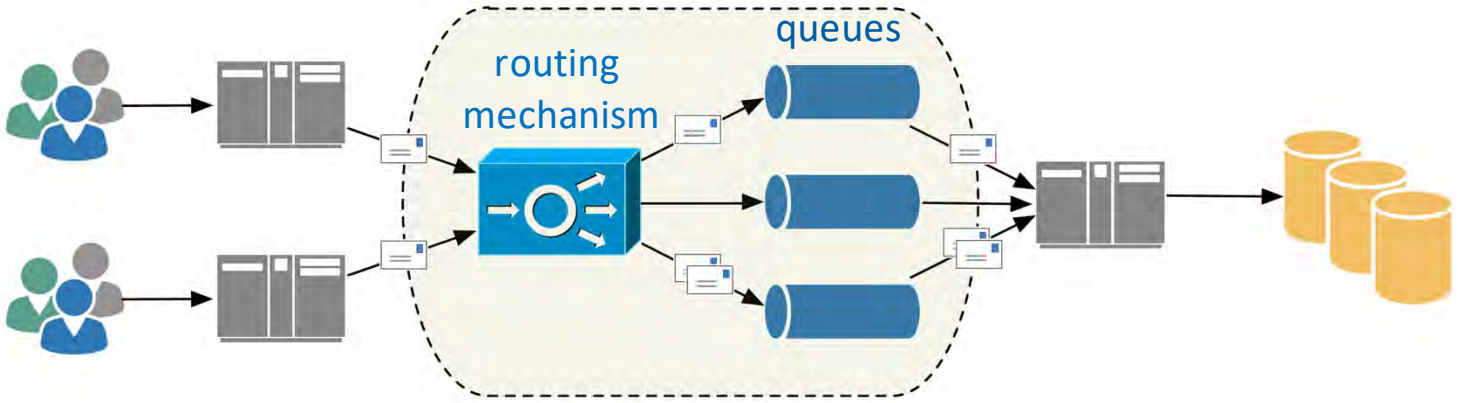
What is messaging?

Publish-subscribe bus



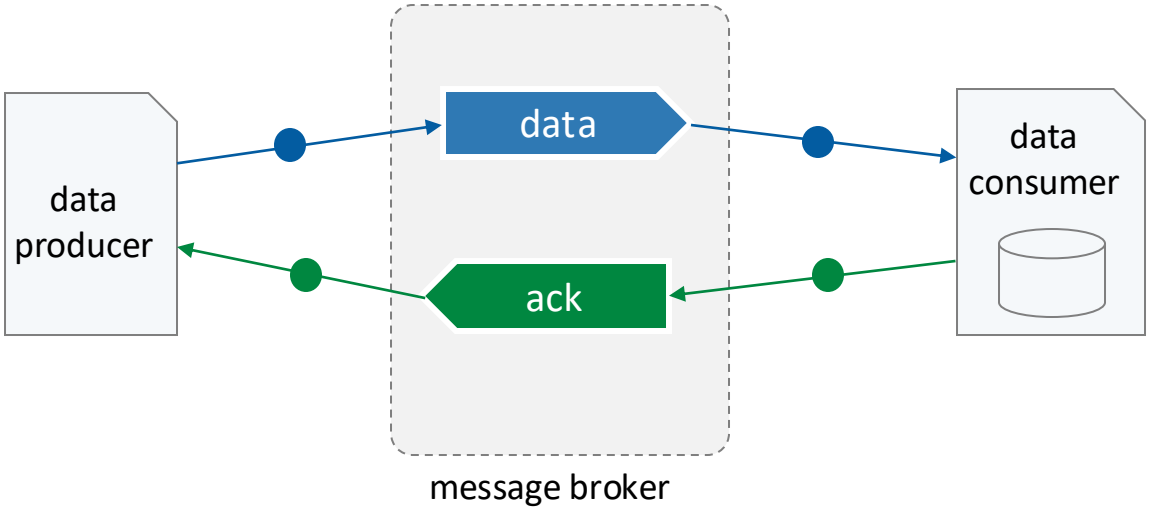
What is messaging?

Main objectives: scaling & extensibility



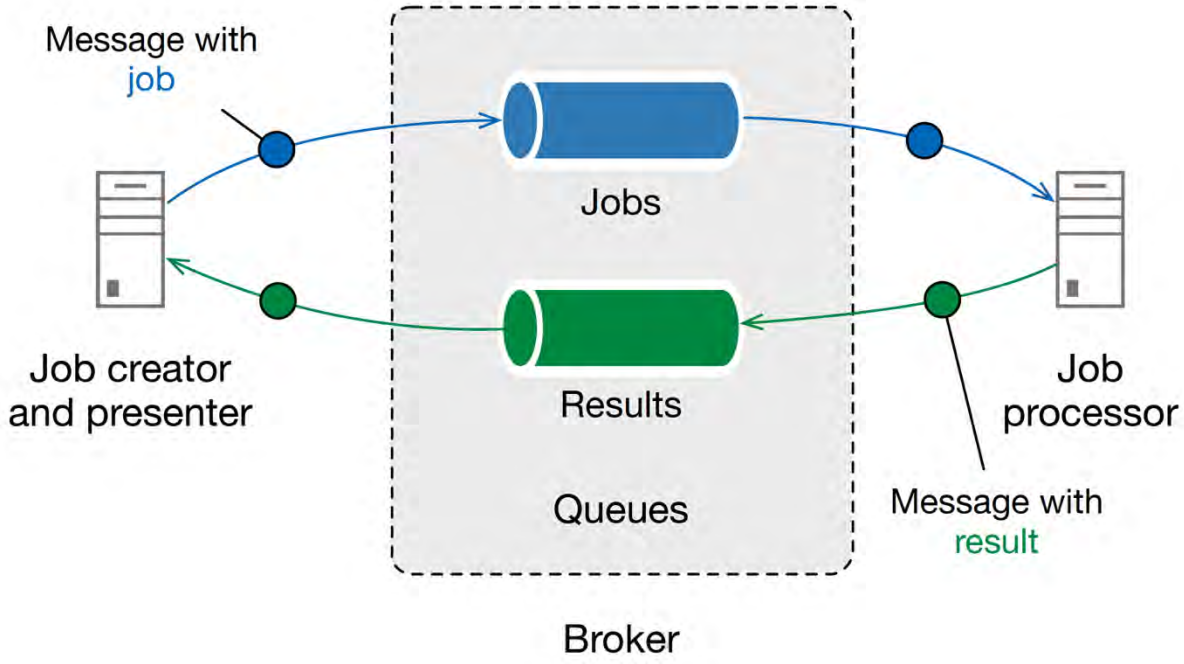
Messaging: use cases

Data communication



Messaging: use cases

Job request



Management of requests to the backend

Load balancing

- ...self-explanatory

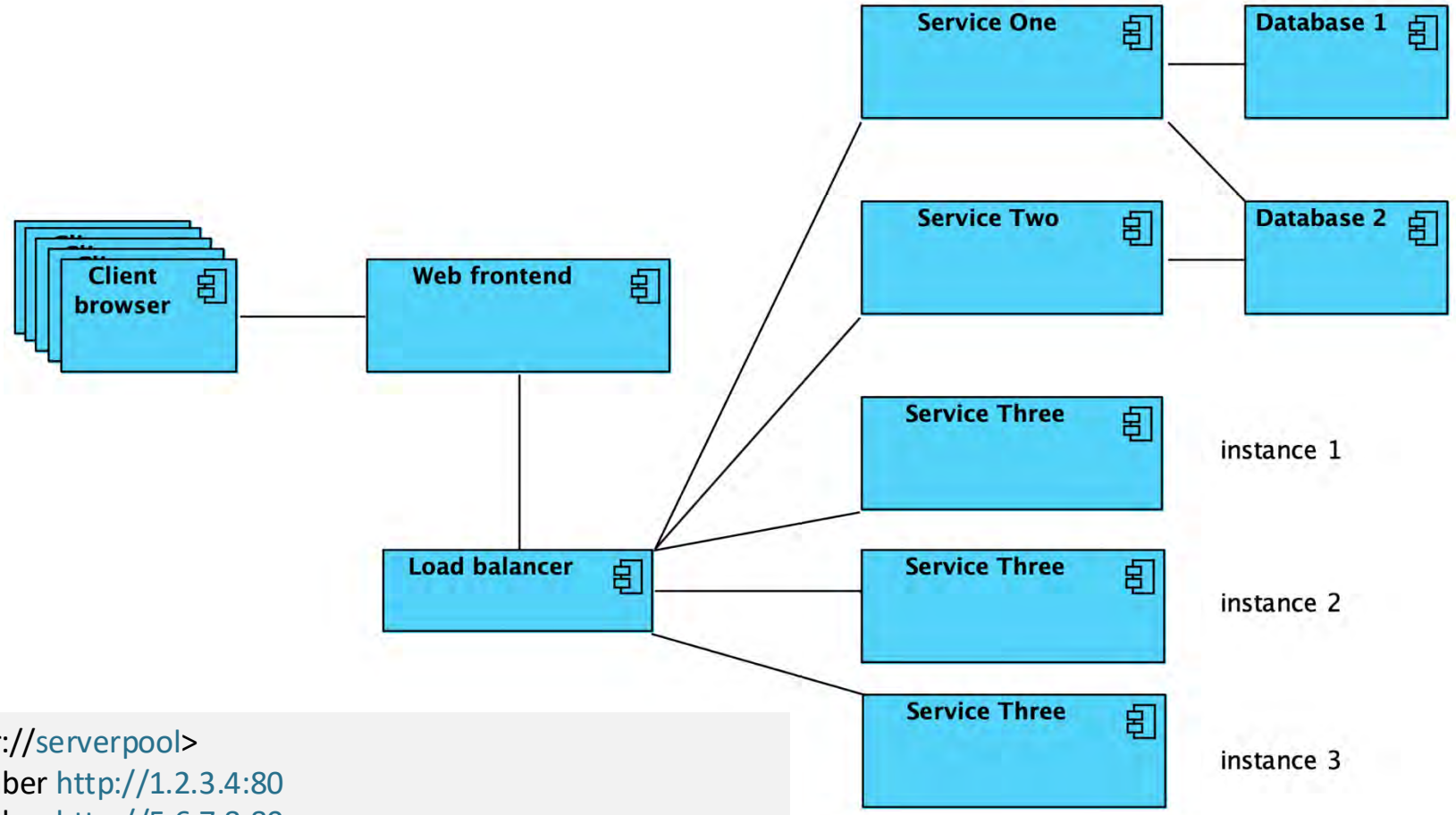
Reverse proxying

- Map ports/services to paths

API gateways

- Plus: quotas, auth, more

Load balancing



Apache

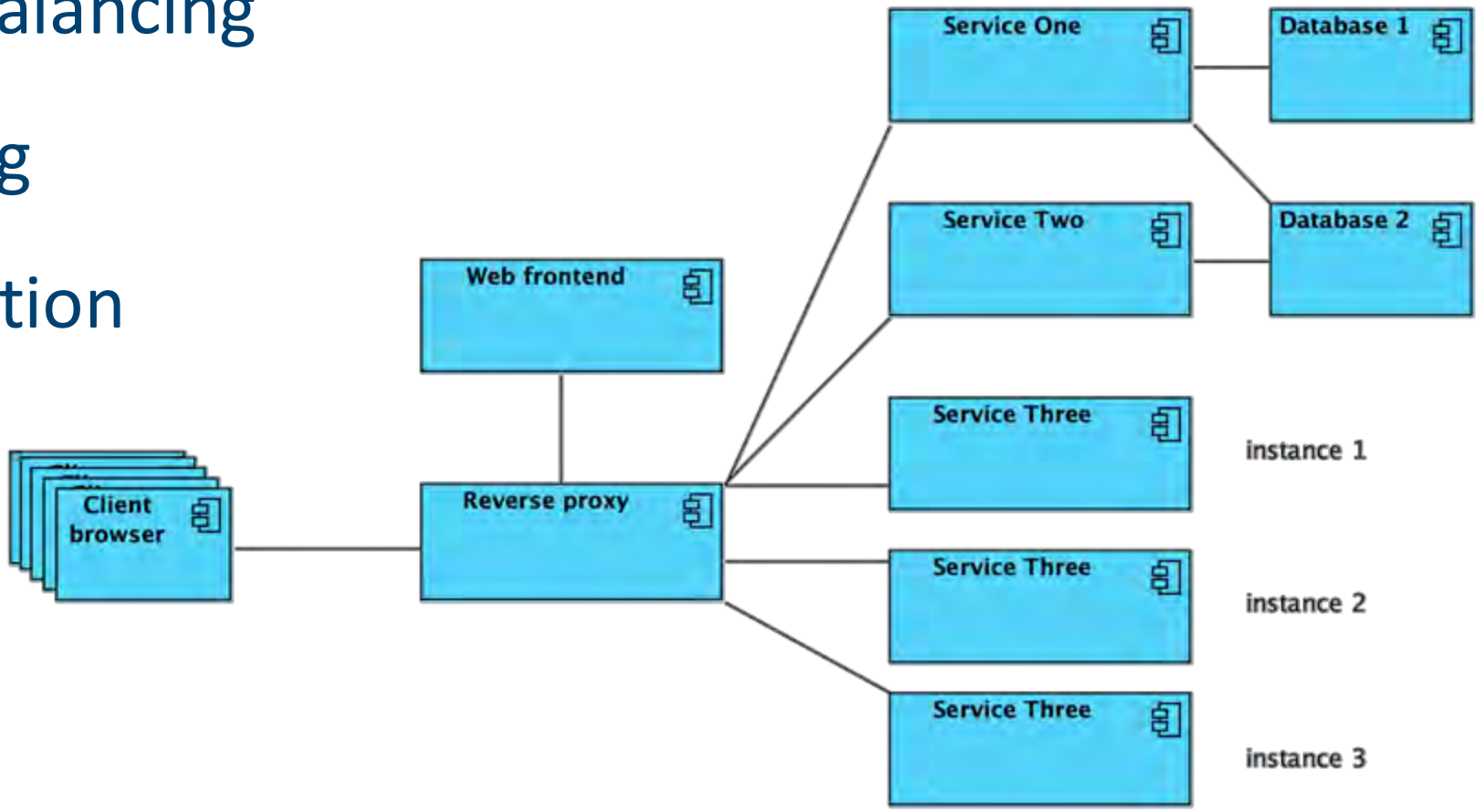
```
<proxy balancer://serverpool>
   BalancerMember http://1.2.3.4:80
   BalancerMember http://5.6.7.8:80
   ProxySet lbmethod=bytraffic
</proxy>
ProxyPass "/site" "balancer://serverpool/"
ProxyPassReverse "/site" "balancer://serverpool/"
```

Reverse proxy

Load balancing

Caching

Encryption



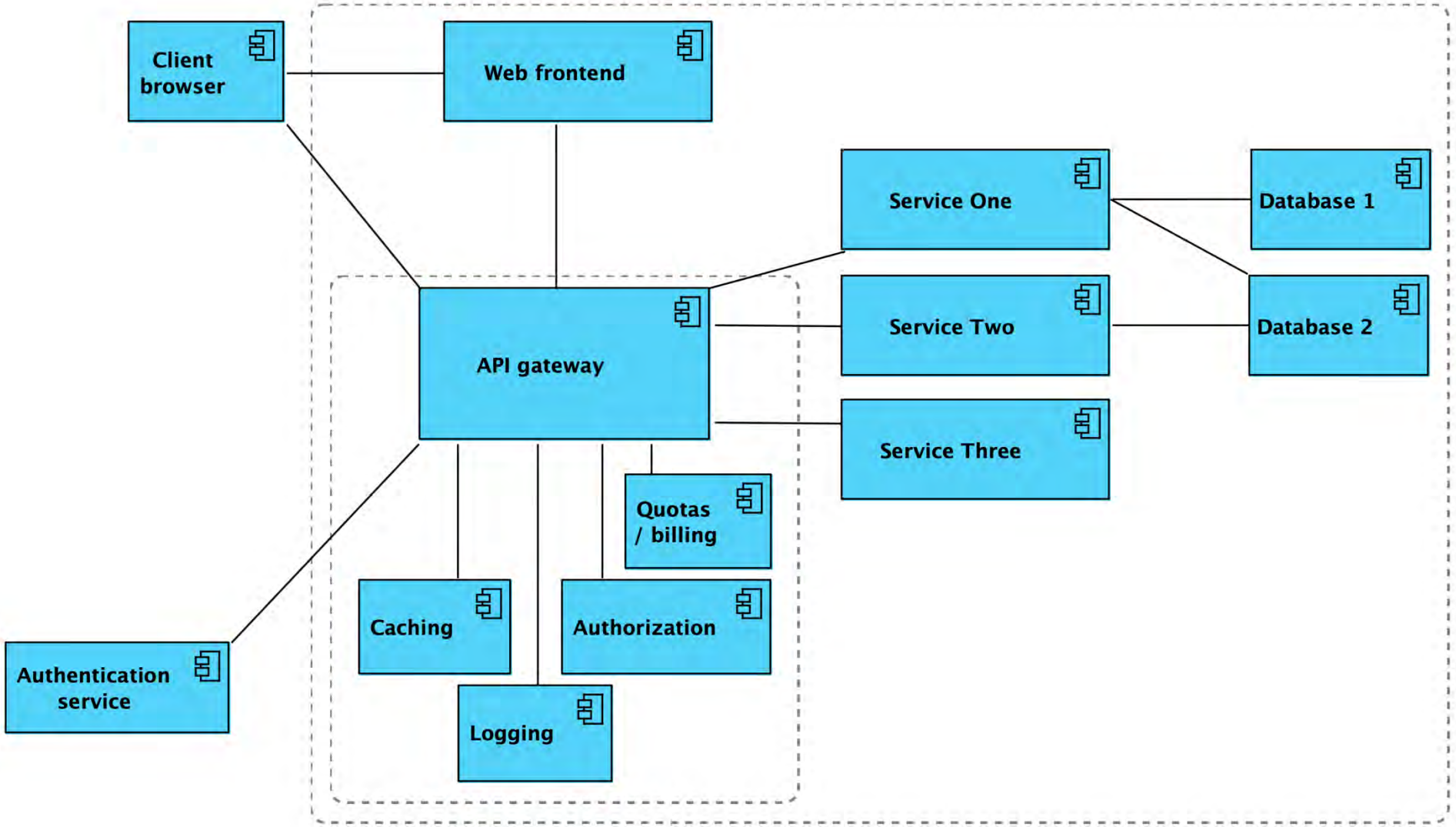
API gateway: "ReverseProxy+++"

API Gateway Services

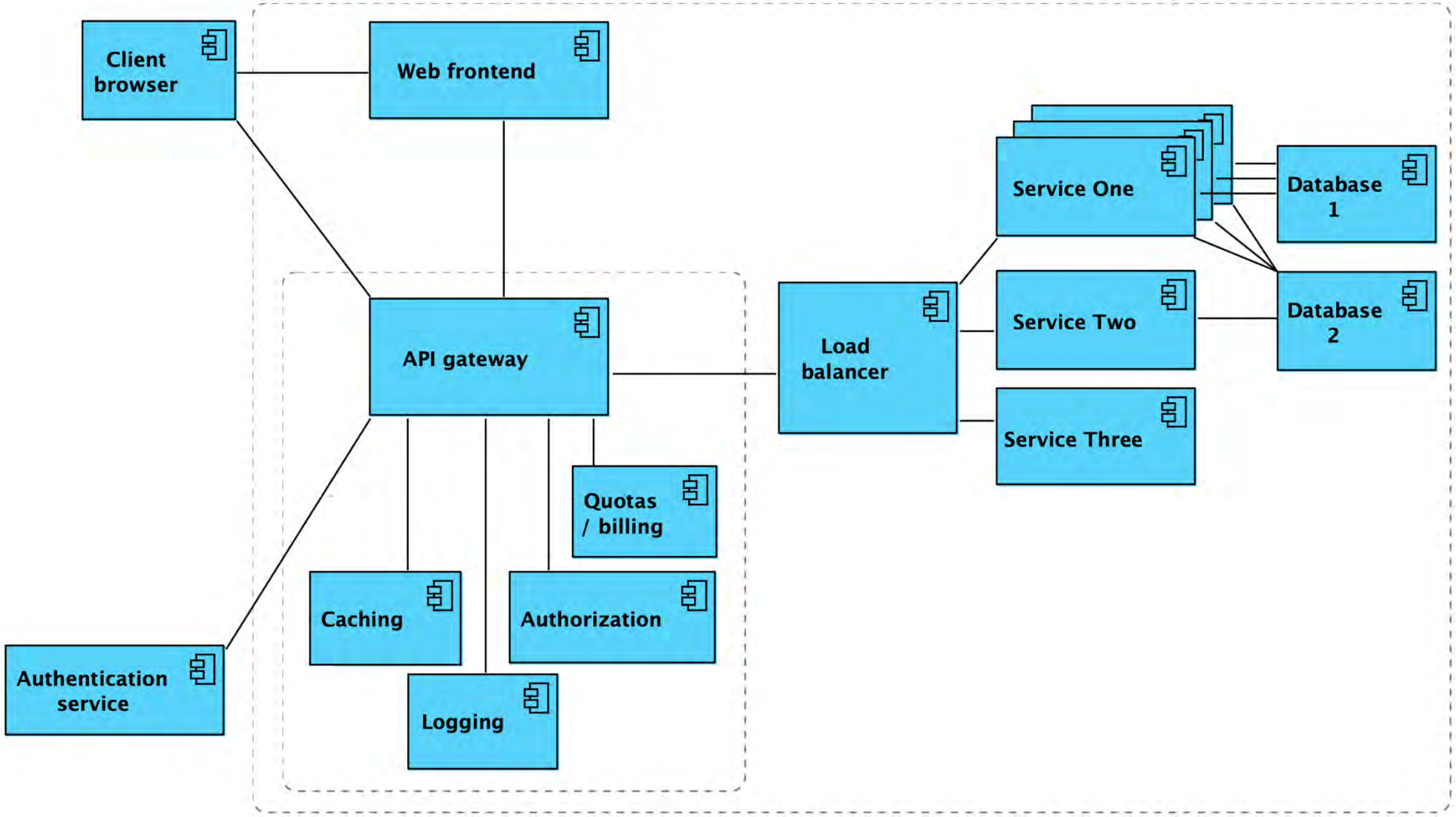
- Authentication
- Authorization
- IP whitelisting
- Quotas
- Logging
- Tracing
- More

Function	Load Balancer	Reverse Proxy	API Gateway
Balance incoming requests	✅	✅	✅
URL Rewrite		✅	✅
Caching		✅	✅
Prevention from Attack			✅
Protocol Translation			✅
Authentication			✅
IP Whitelisting			✅
Rate Limiting, Quota			✅
Logging			✅
Tracing			✅
Health Checking			✅
Dynamic Config Updates			✅

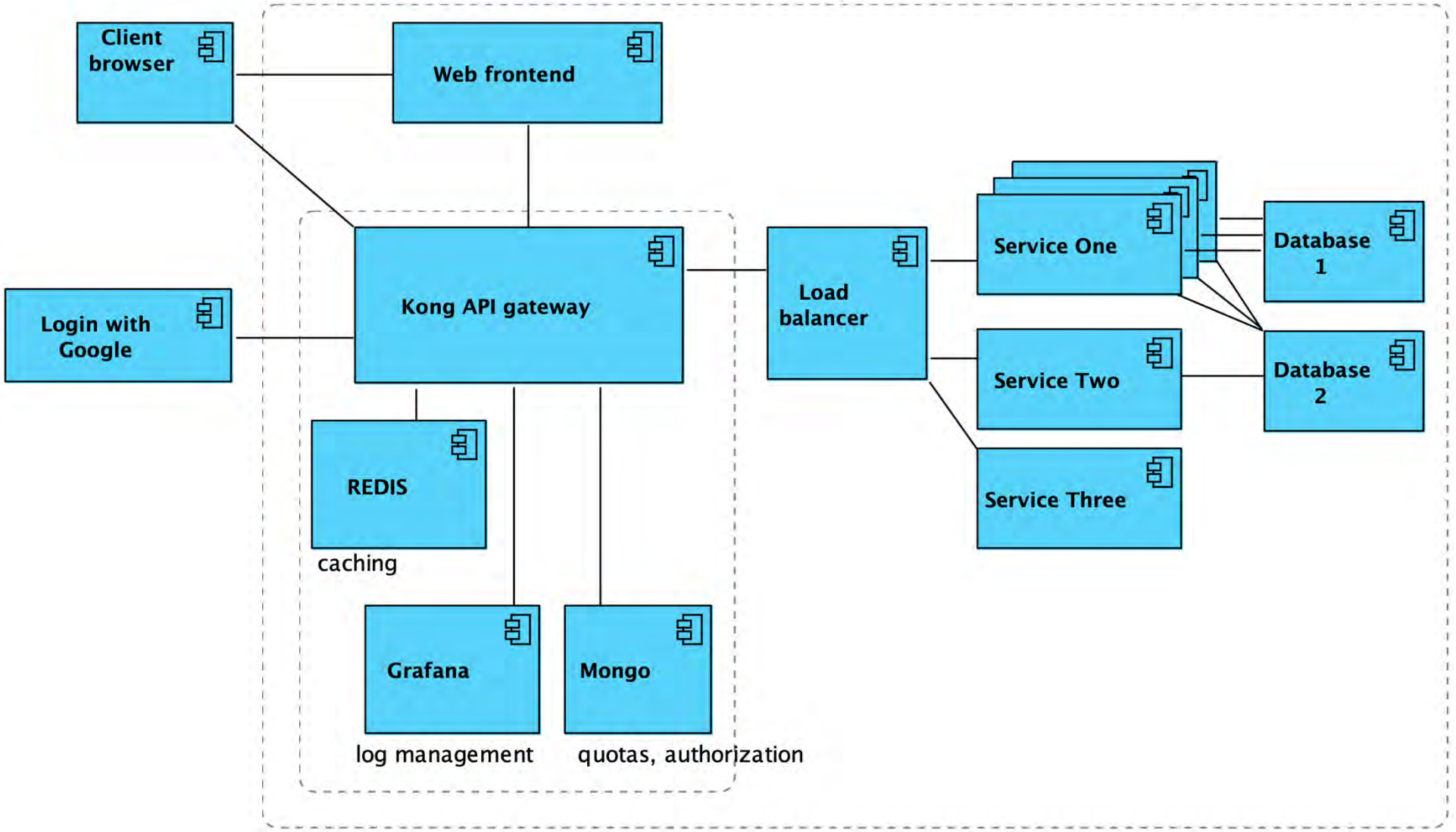
API gateways



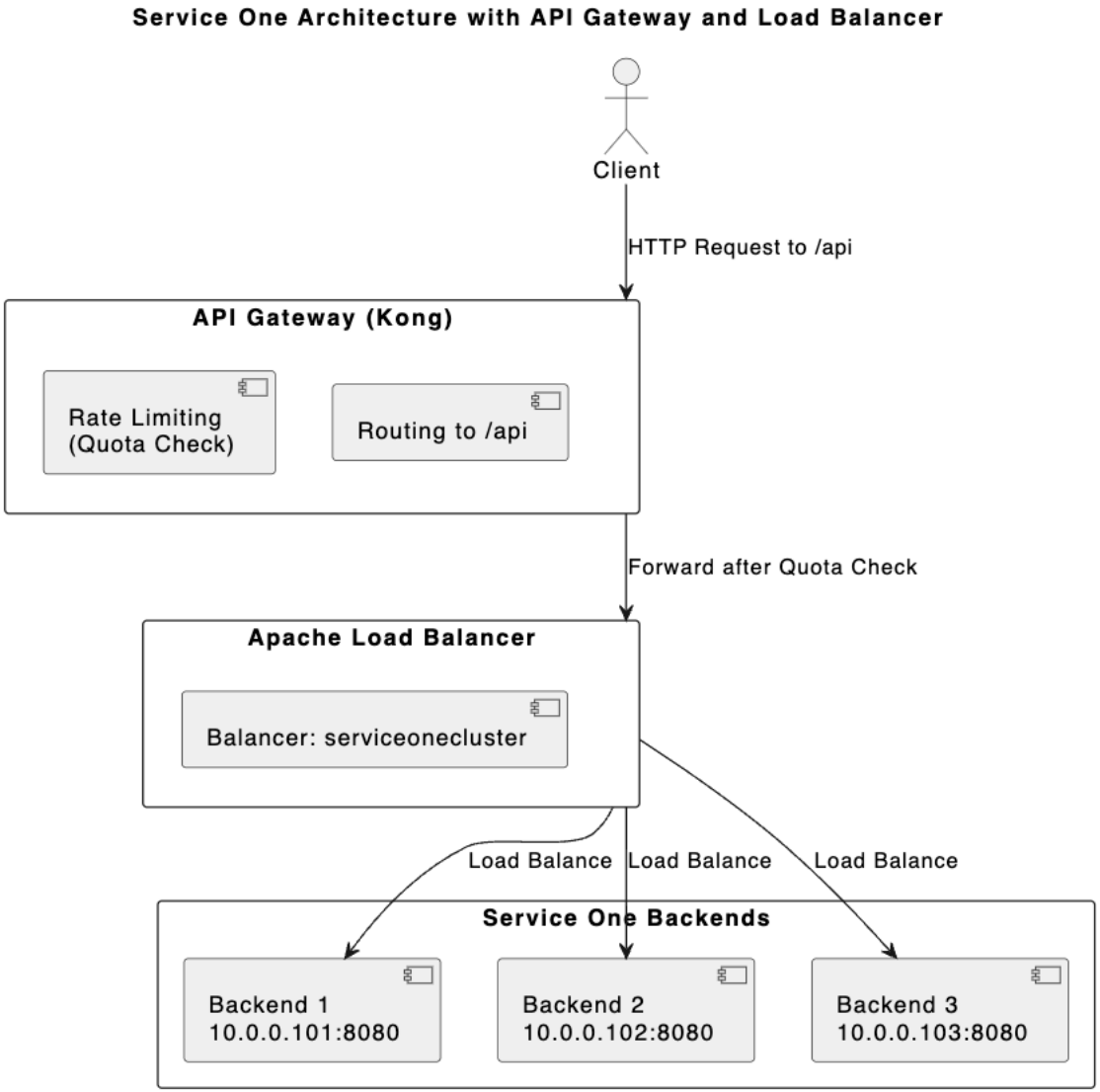
API gateways



API gateways - Kong



API gateways - Kong



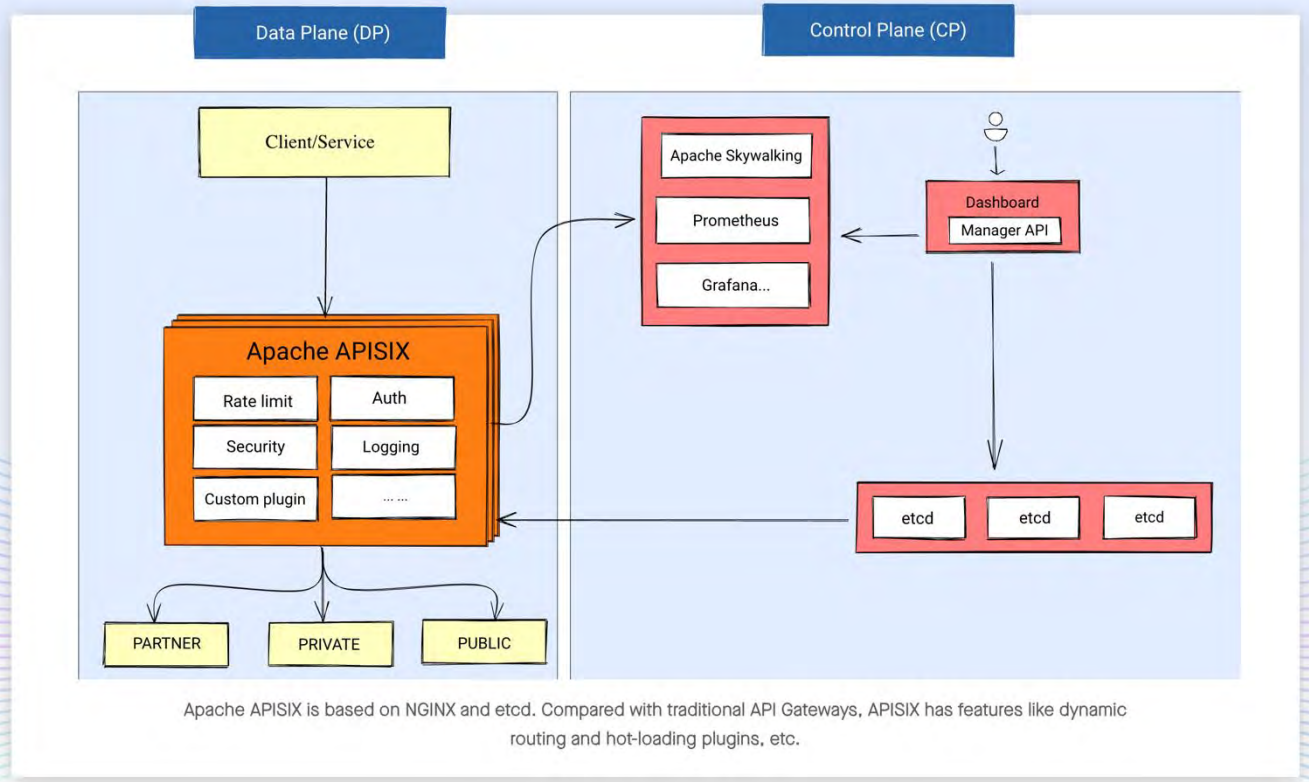
API gateways – Apache APISIX

Apache APISIX®

Docs Blog Case Studies Downloads Help Team Resources Aあ English Search

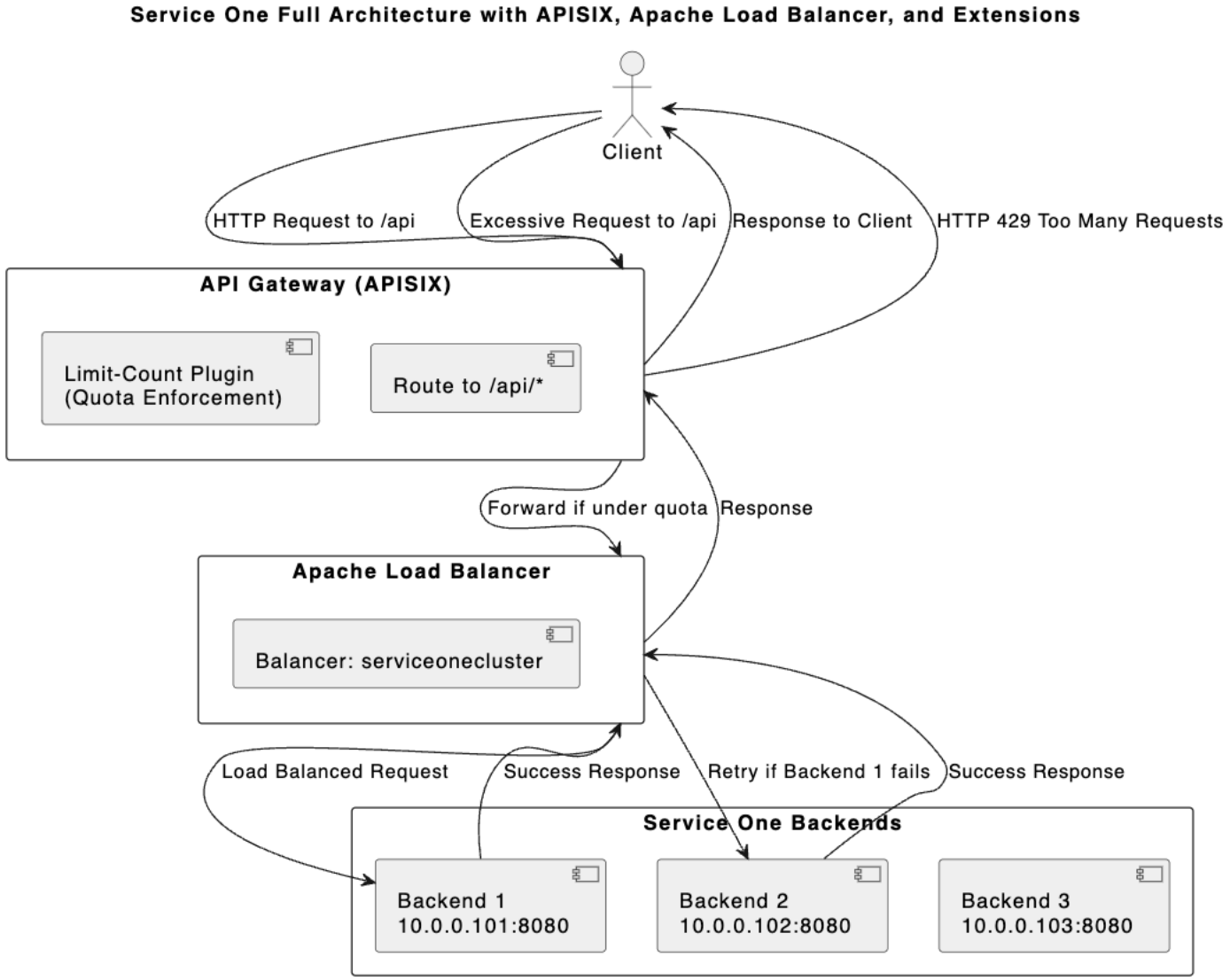
Building for large-scale, high value systems

Apache APISIX provides open source API Gateway to help you manage microservices, delivering the ultimate performance, security, and scalable platform for all your APIs and microservices.

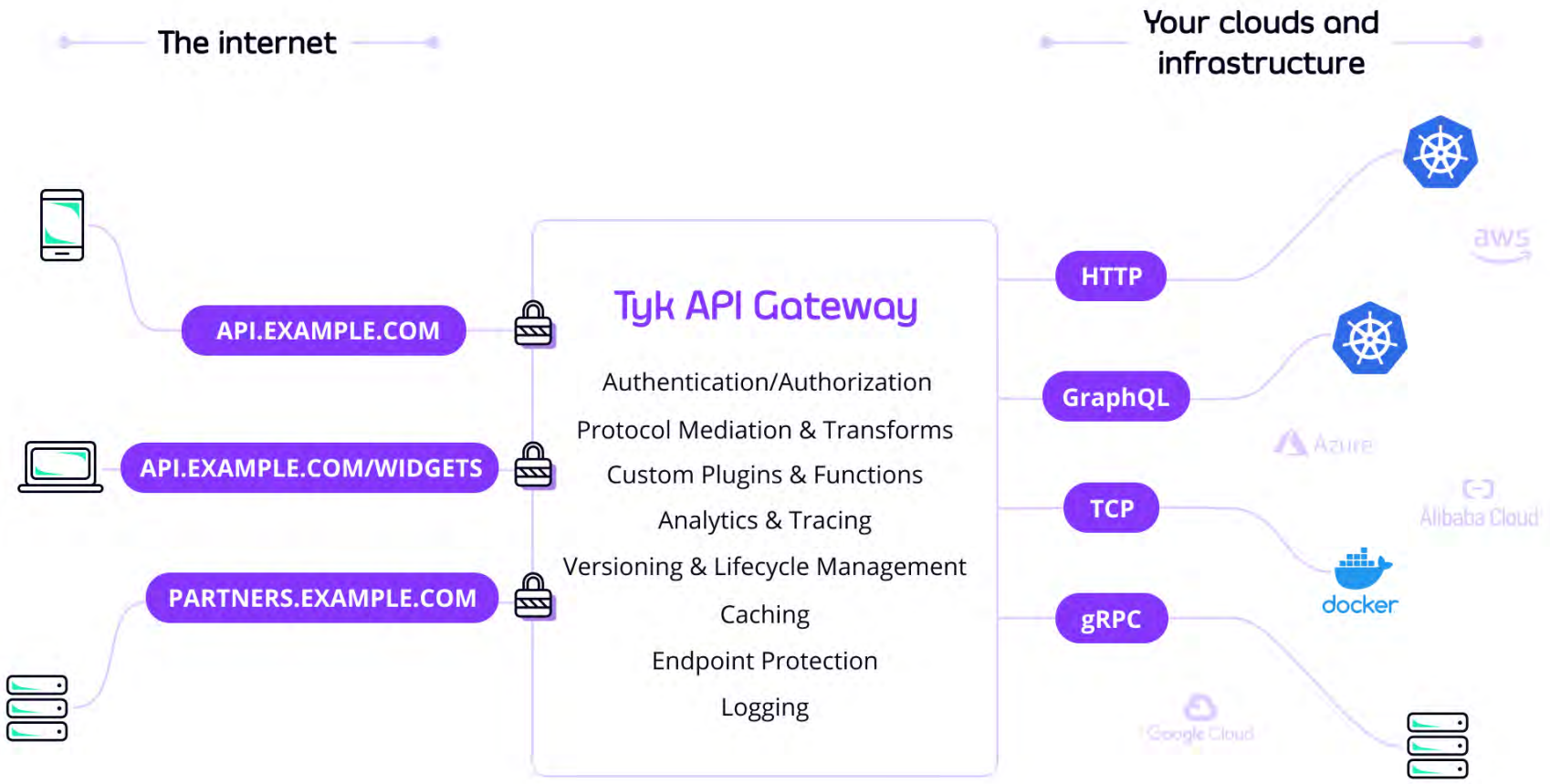


Apache APISIX is based on NGINX and etcd. Compared with traditional API Gateways, APISIX has features like dynamic routing and hot-loading plugins, etc.

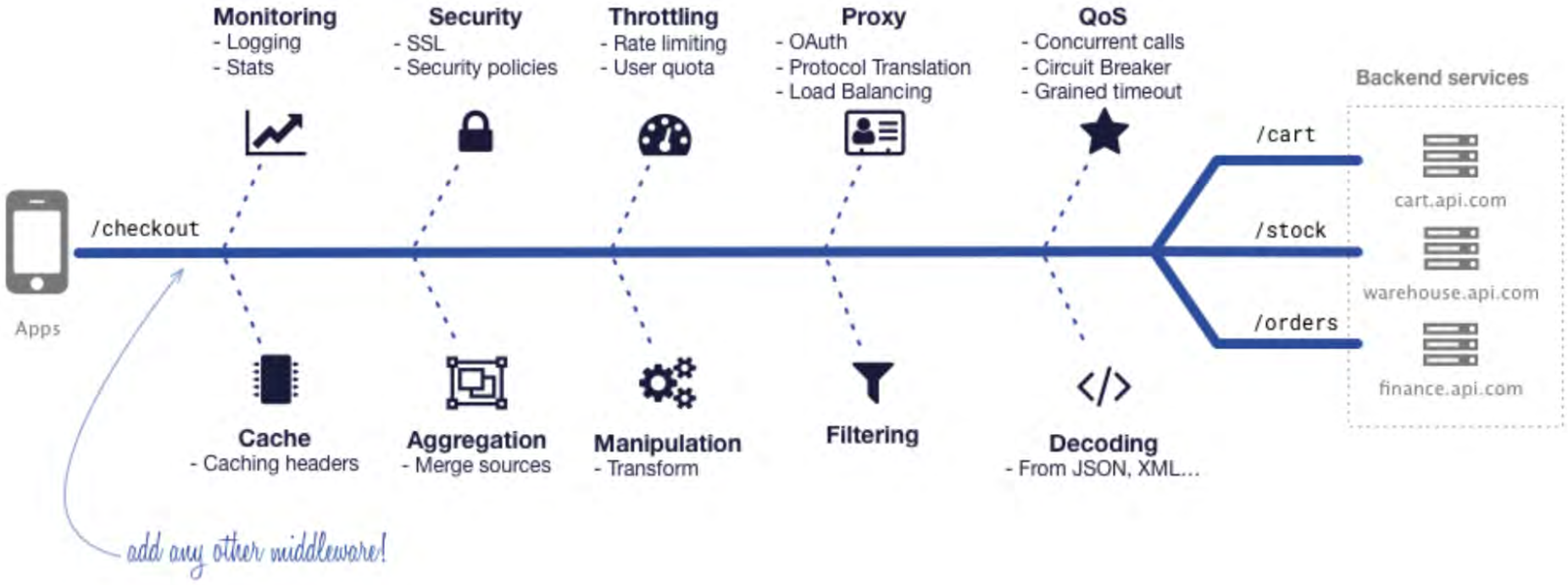
API gateways – Apache APISIX



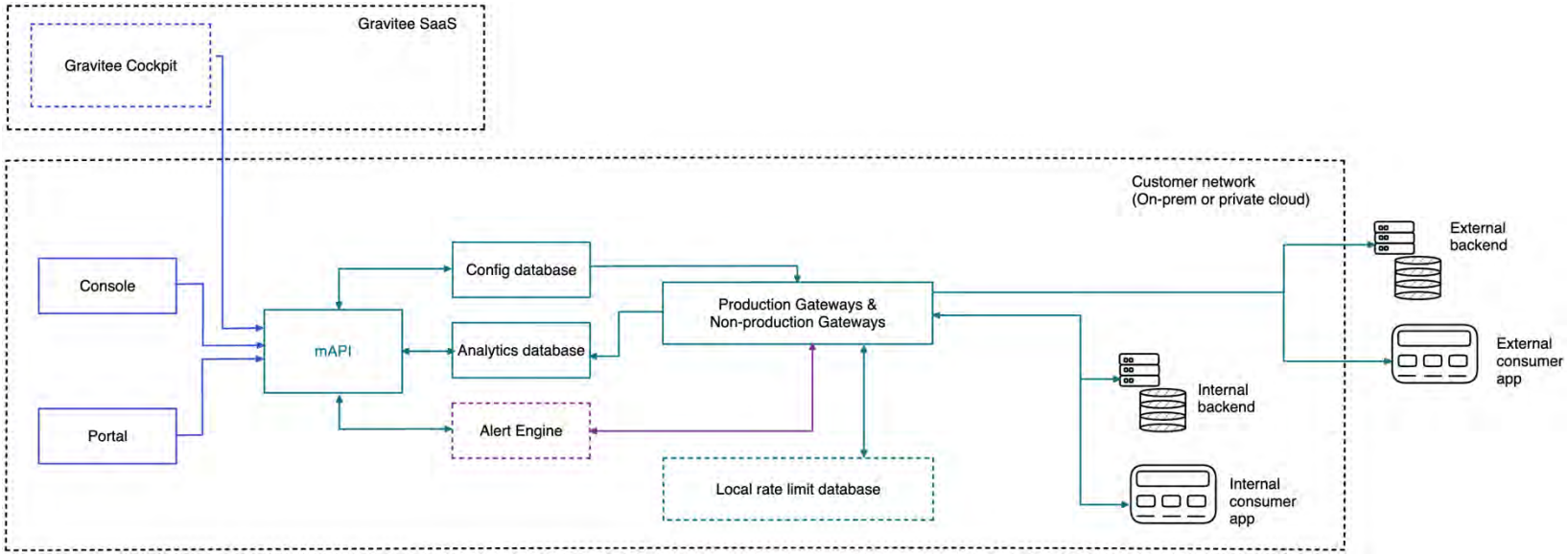
API gateways – Tyk



API gateways – Tyk



Gravitee open source

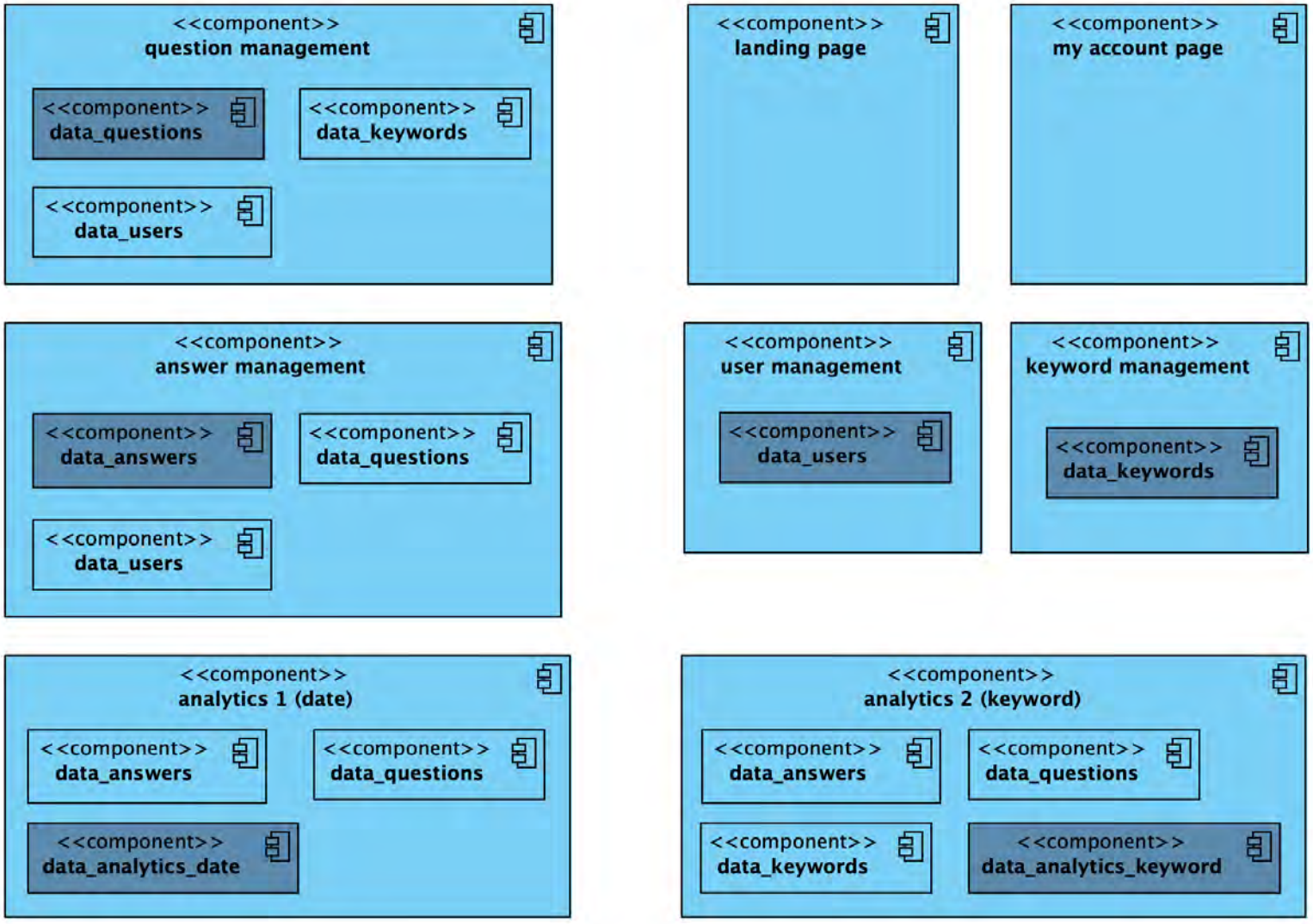


API gateways and microservices

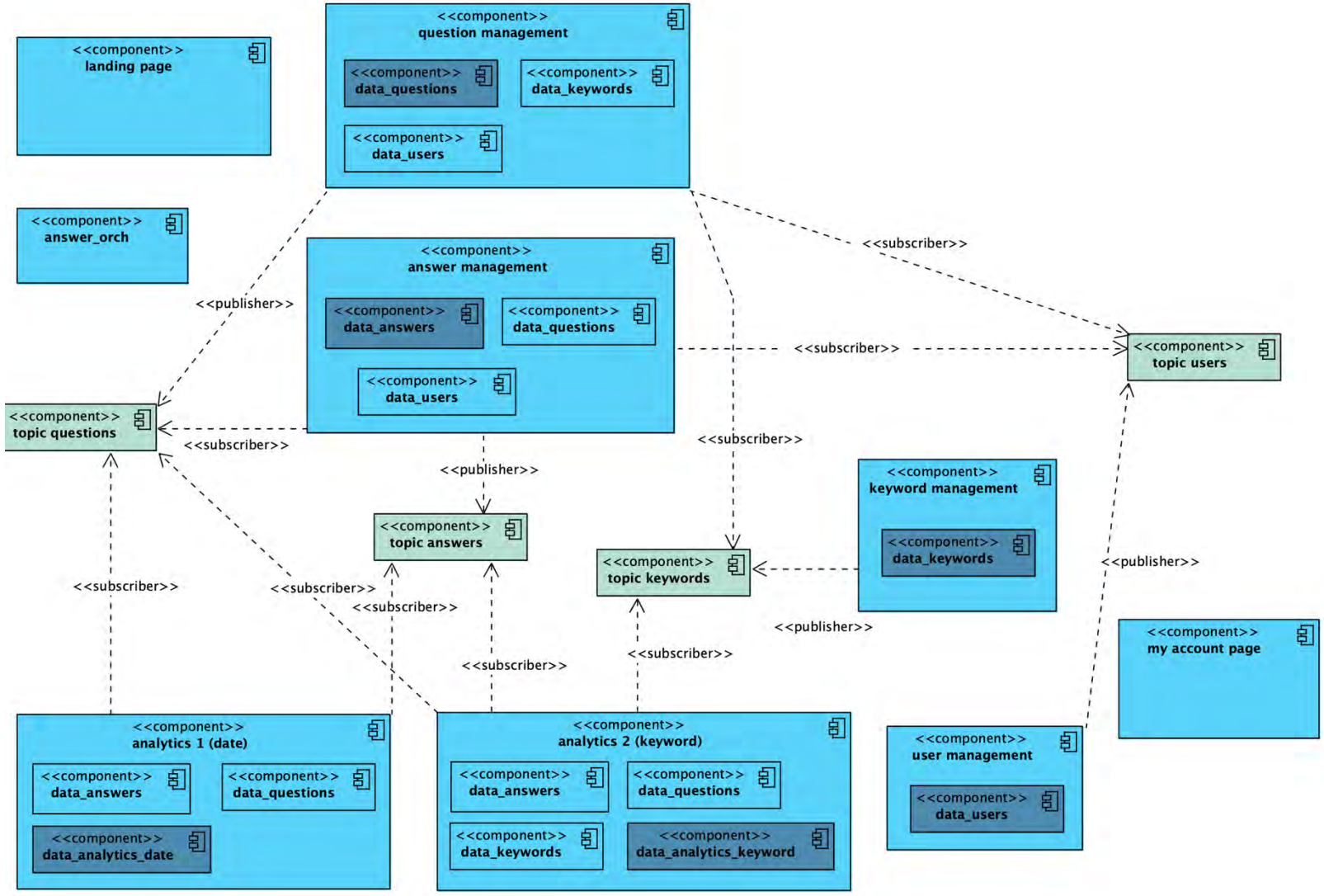
Advantages

- Isolate microservice clients from microservices
- Mimimize impact on clients when updating microservice implementations
- Allow each microservice in many applications

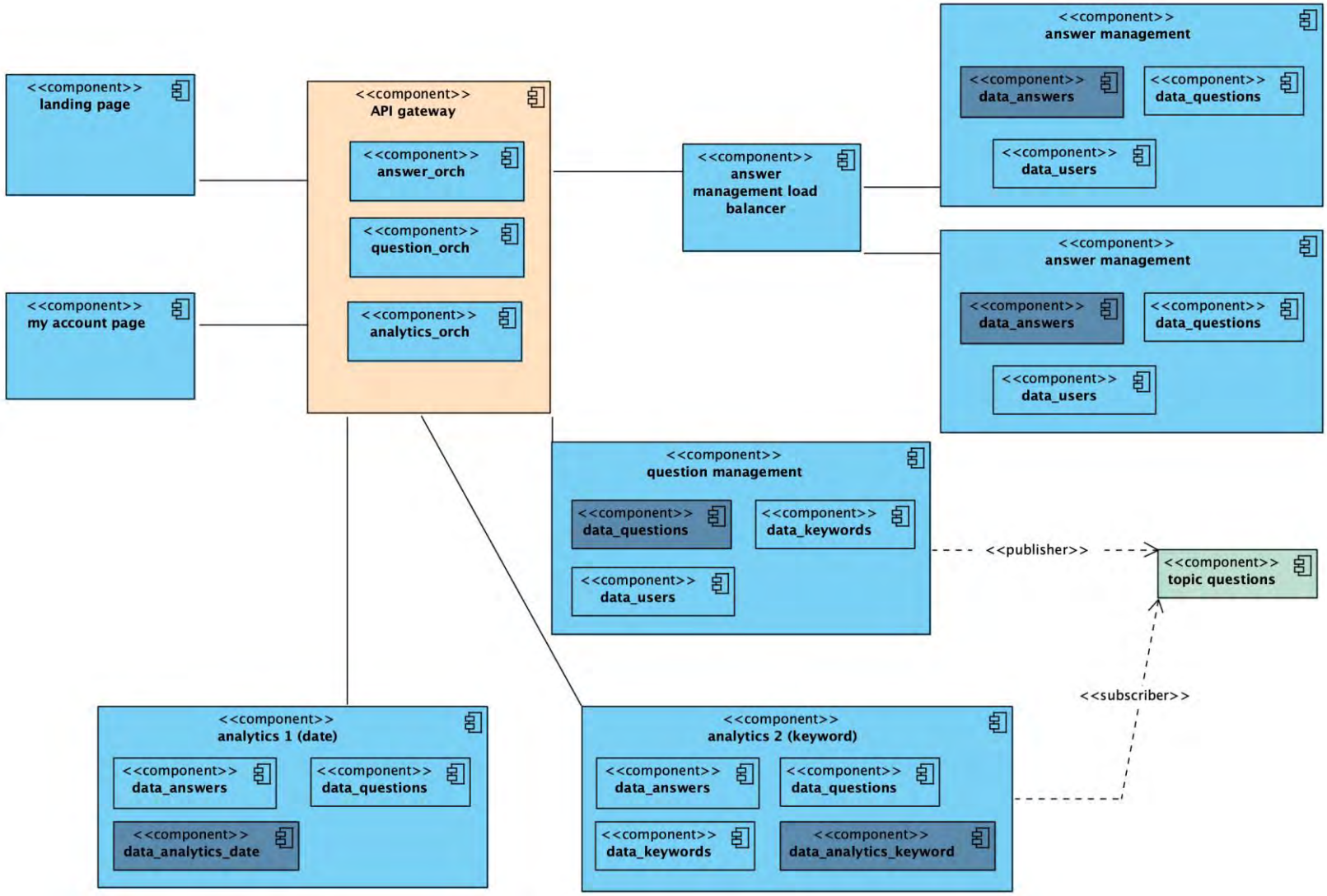
Putting it all together



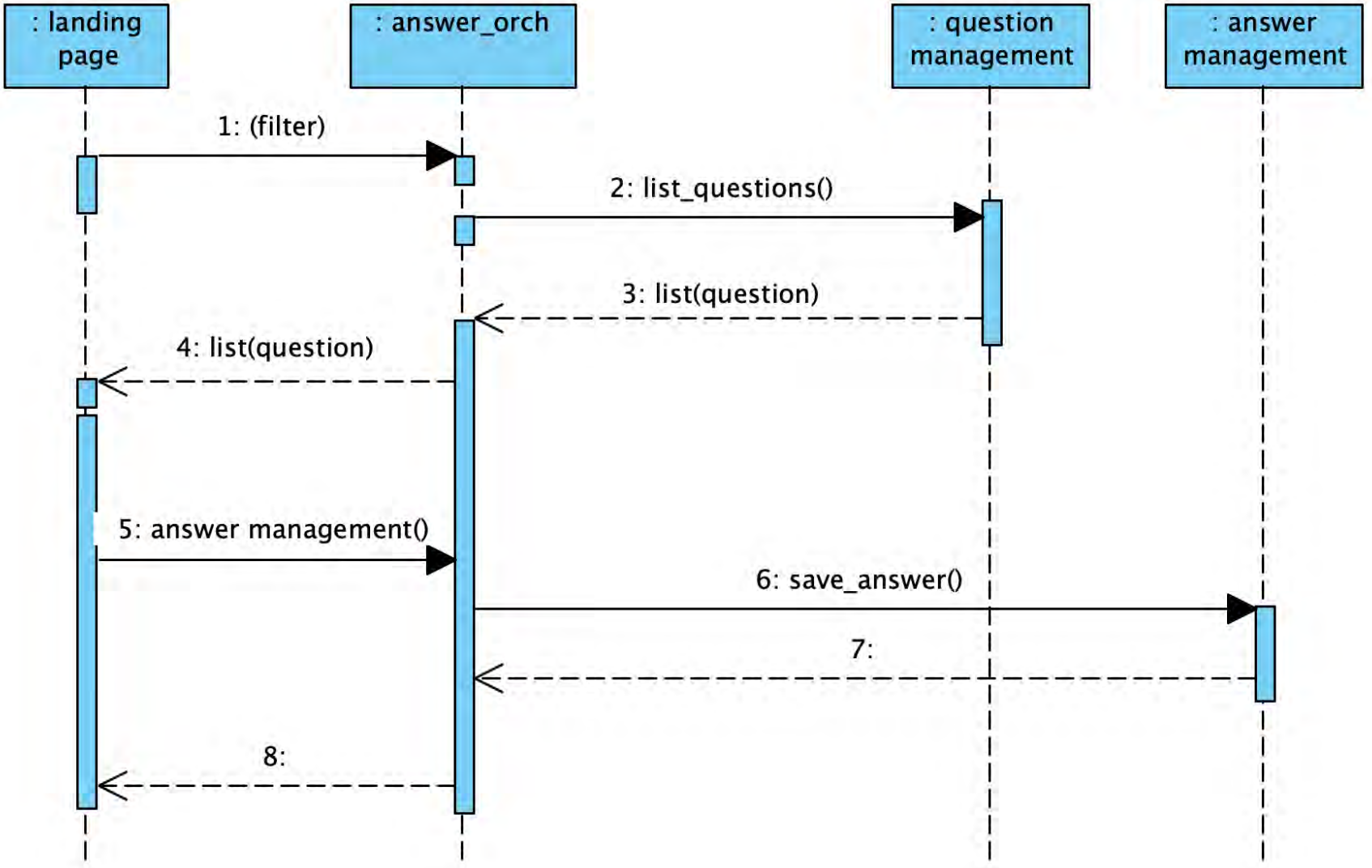
Putting it all together



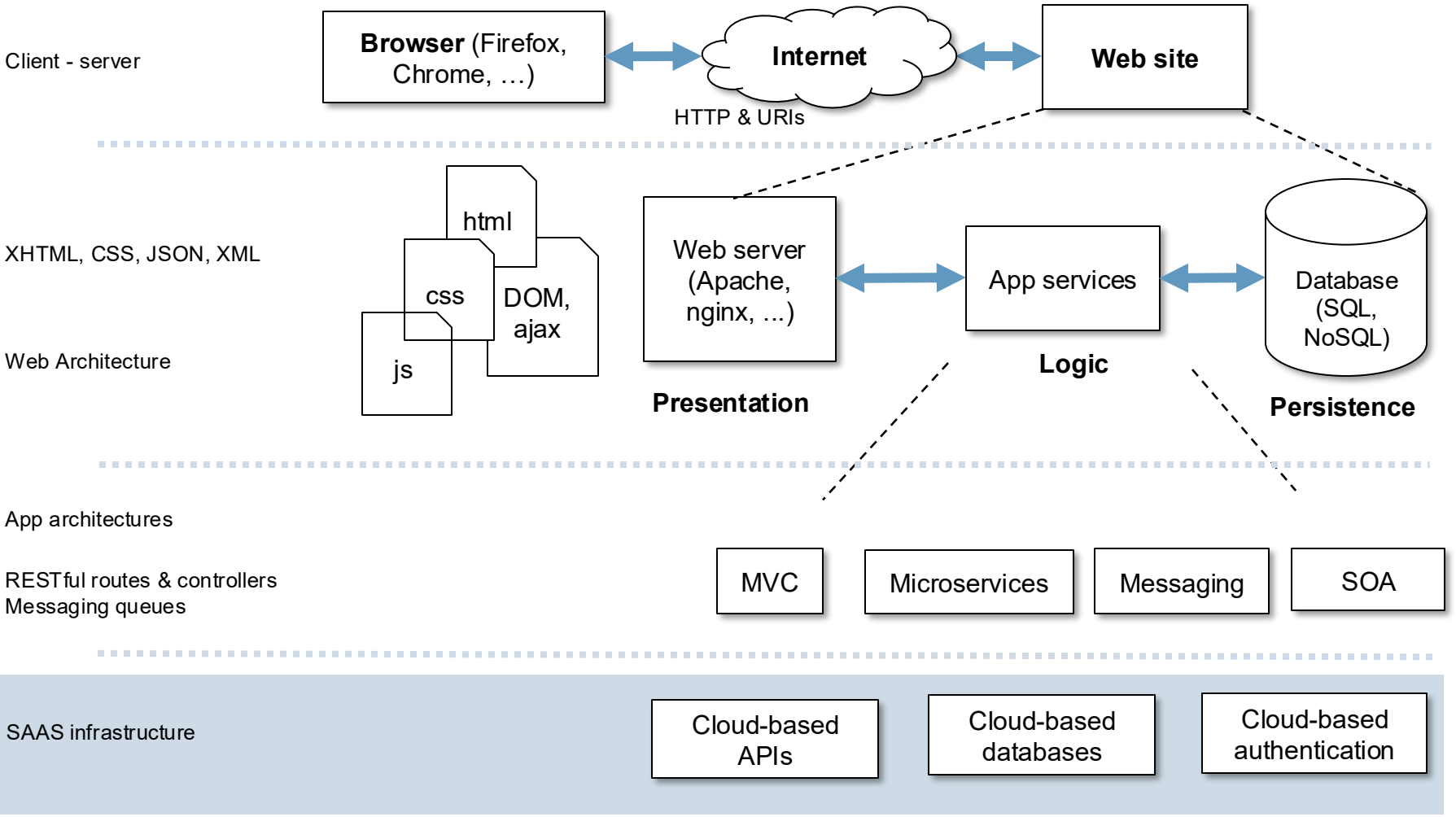
Putting it all together



Putting it all together

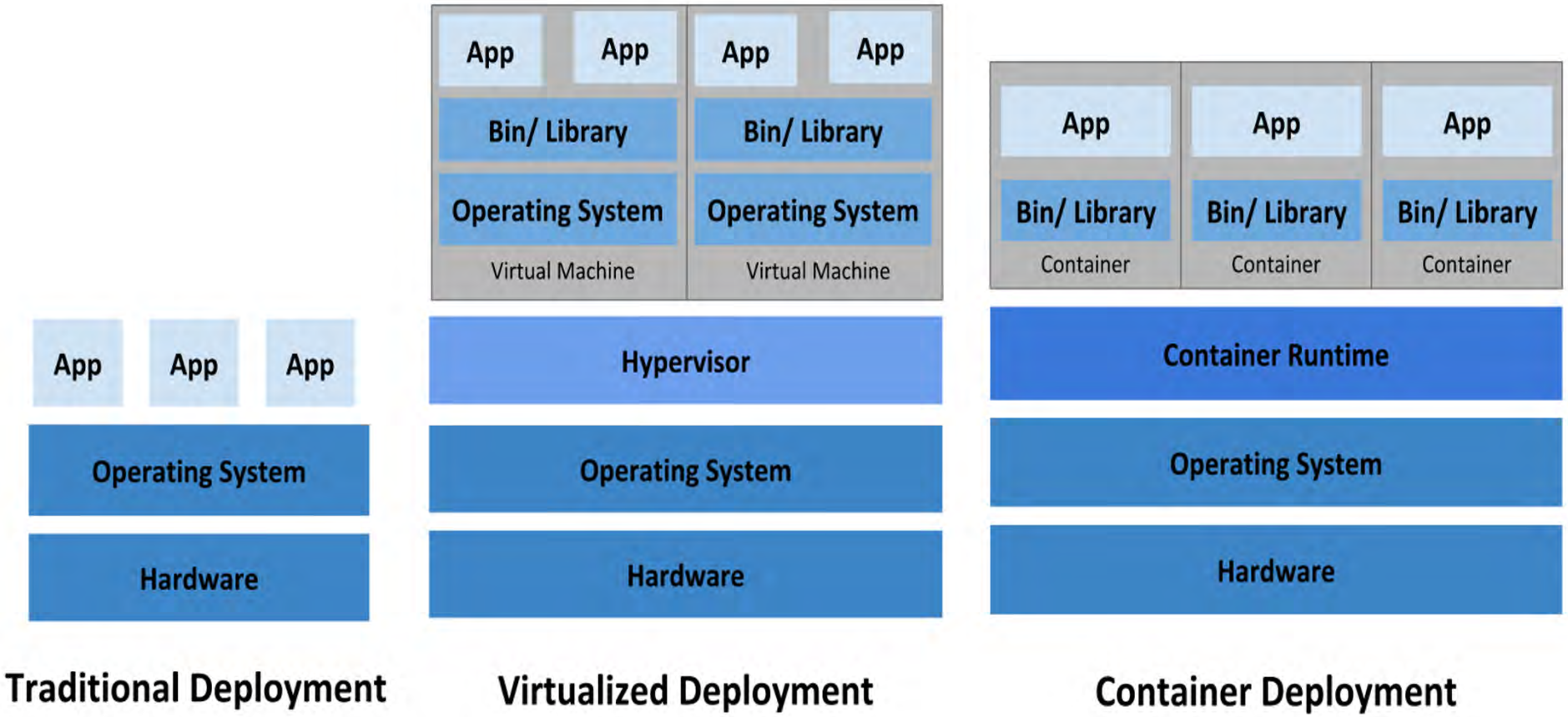


Infrastructure

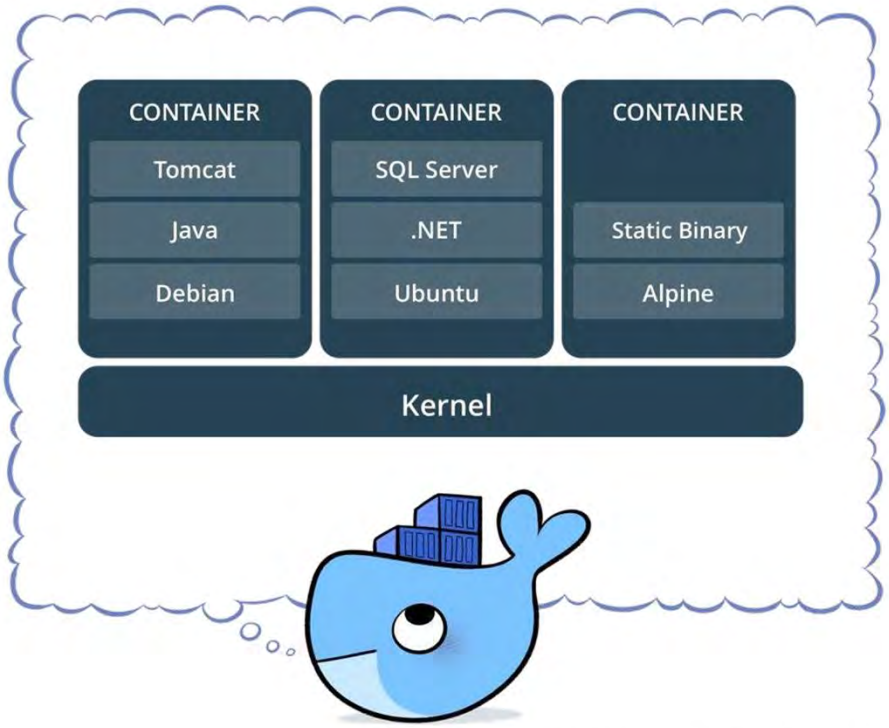


Infrastructures for execution of SaaS apps

Three Deployment Generations



Containers



(source:Alexander González)

Standardized packaging for software and dependencies

Isolate apps from each other

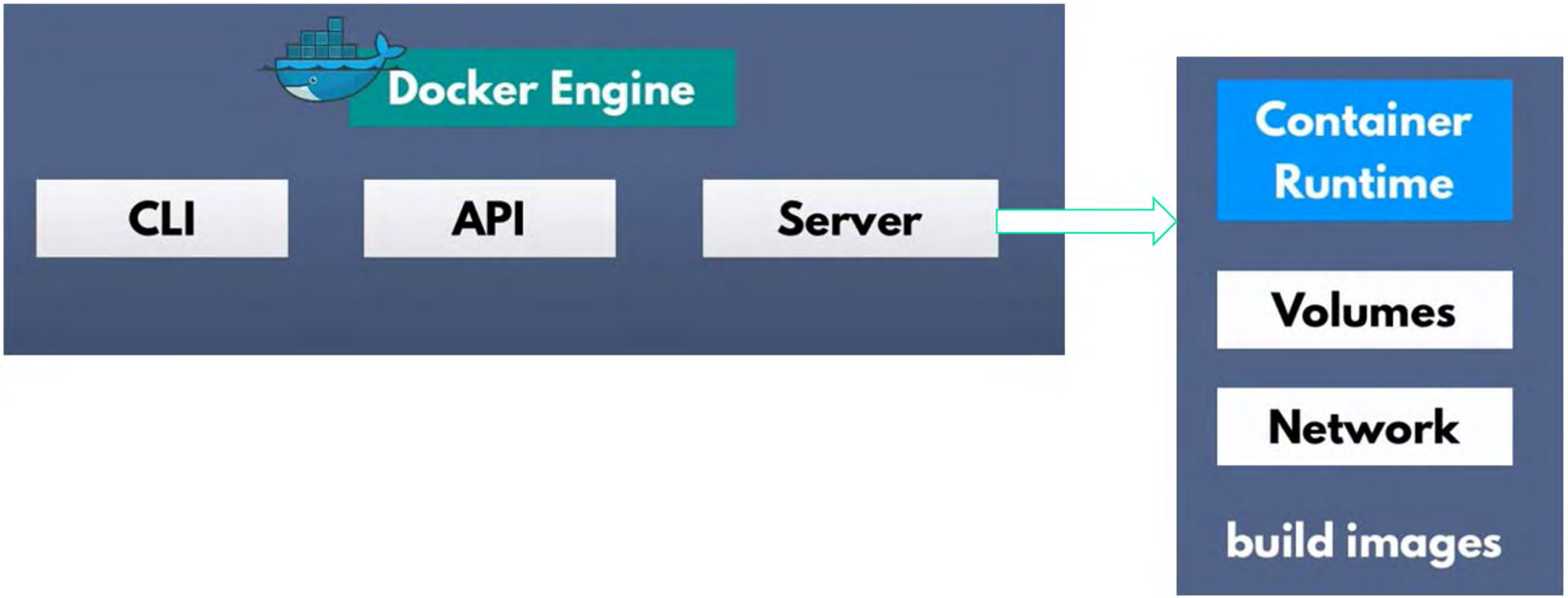
Share the **same** OS kernel

Works for **all** major **Linux** distributions and **Windows**

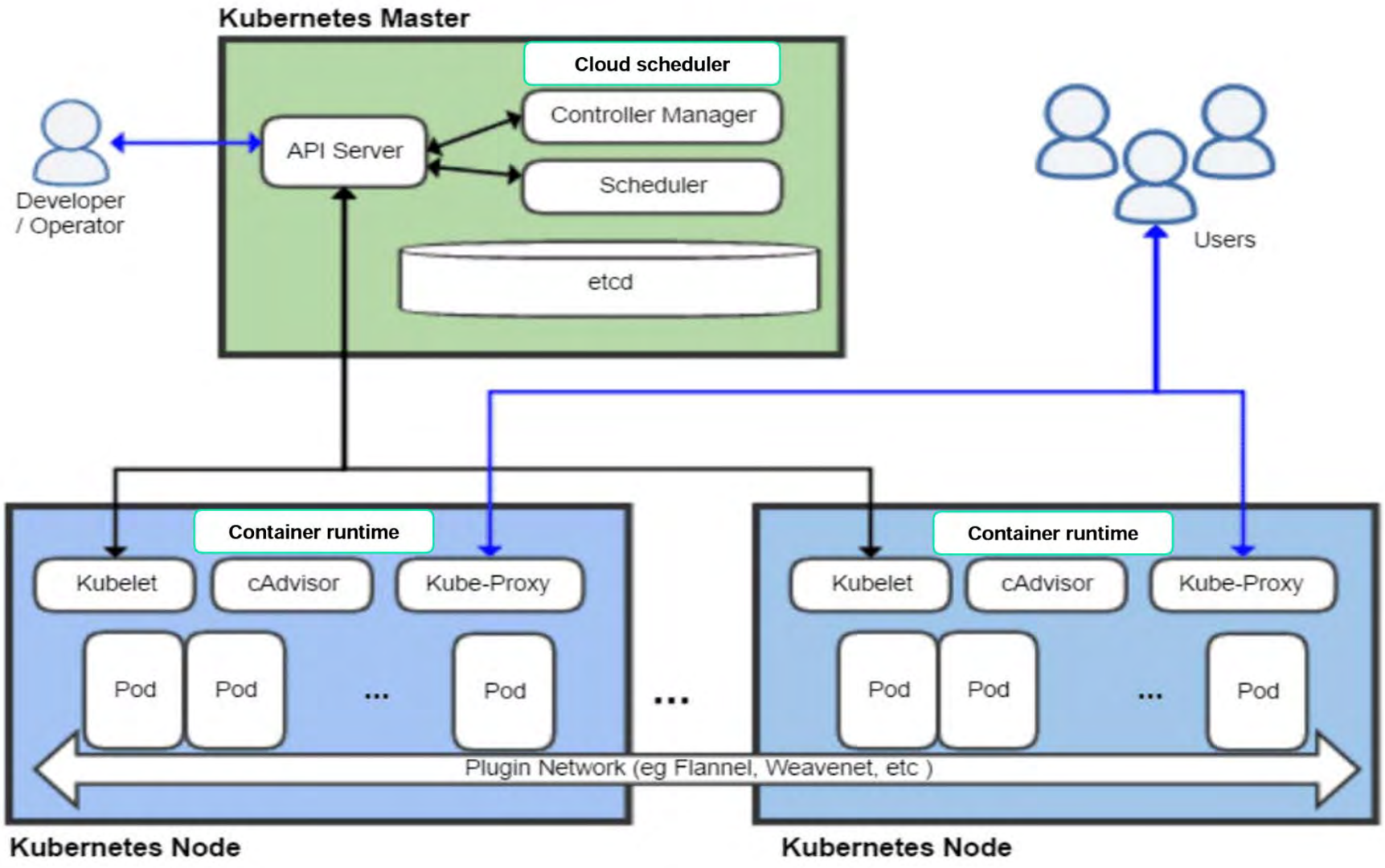
Lightweight, open, secure platform

Simplify **building**, **shipping**, **running** apps

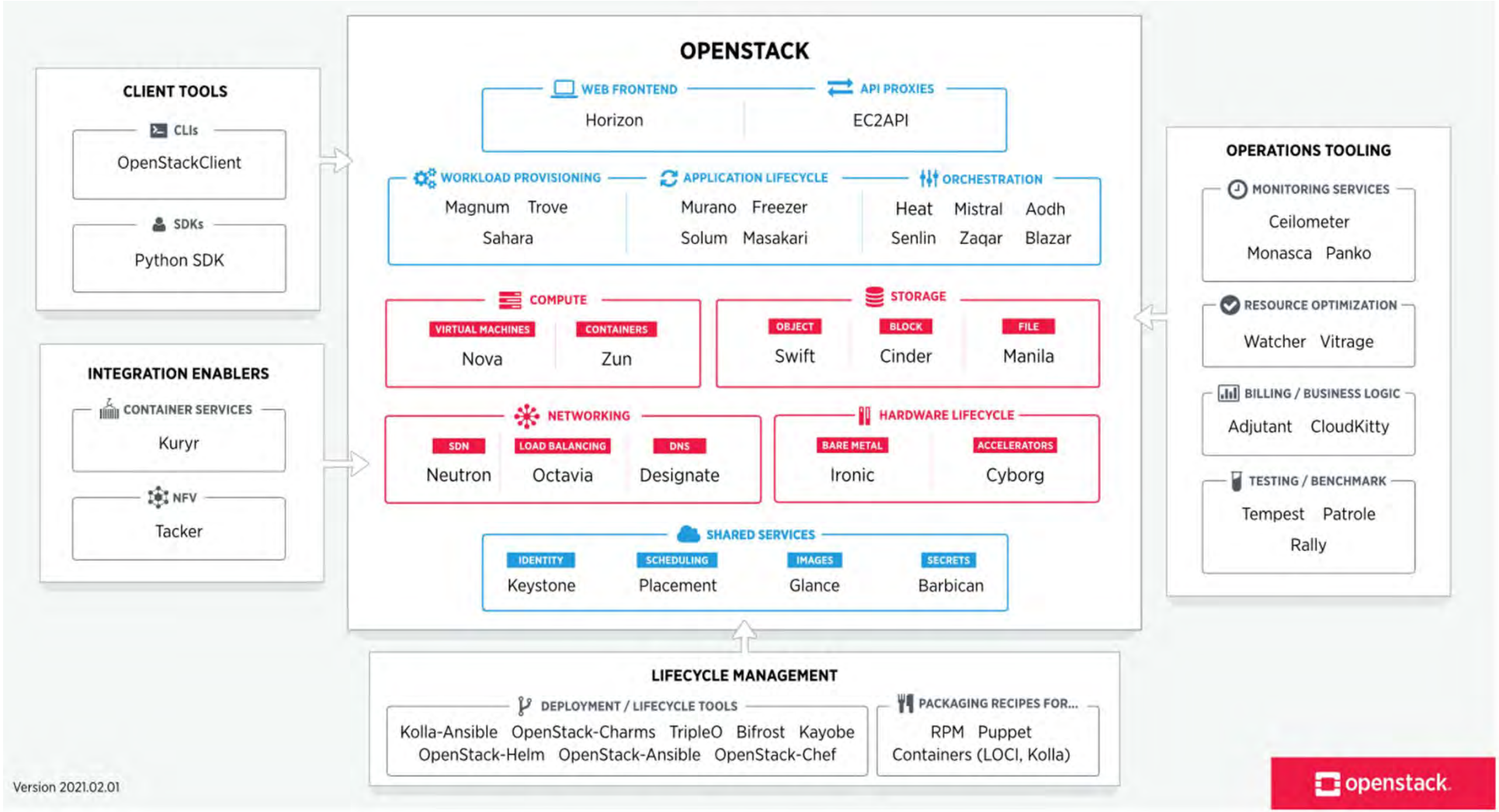
Docker


Docker Engine
CLI
API
Server
Container Runtime
Volumes
Network
build images

Kubernetes


Kubernetes Master
Cloud scheduler
Controller Manager
API Server
Scheduler
etcd
Developer
/ Operator
Users
Container runtime
Kubelet
cAdvisor
Kube-Proxy
Pod
Pod
...
Pod
...
Container runtime
Kubelet
cAdvisor
Kube-Proxy
Pod
Pod
...
Pod
Plugin Network (eg Flannel, Weavenet, etc)
Kubernetes Node
Kubernetes Node

Openstack


CLIENT TOOLS
CLIs
OpenStackClient
SDKs
Python SDK
INTEGRATION ENABLERS
CONTAINER SERVICES
Kuryr
NFV
Tacker
OPENSTACK
WEB FRONTEND
Horizon
API PROXIES
EC2API
WORKLOAD PROVISIONING
Magnum Trove
Sahara
APPLICATION LIFECYCLE
Murano Freezer
Solum Masakari
ORCHESTRATION
Heat Mistral Aodh
Senlin Zaqar Blazar
COMPUTE
VIRTUAL MACHINES
Nova
CONTAINERS
Zun
STORAGE
OBJECT
Swift
BLOCK
Cinder
FILE
Manila
NETWORKING
SDN
Neutron
LOAD BALANCING
Octavia
DNS
Designate
HARDWARE LIFECYCLE
BARE METAL
Ironic
ACCELERATORS
Cyborg
SHARED SERVICES
IDENTITY
Keystone
SCHEDULING
Placement
IMAGES
Glance
SECRETS
Barbican
OPERATIONS TOOLING
MONITORING SERVICES
Ceilometer
Monasca Panko
RESOURCE OPTIMIZATION
Watcher Vitrage
BILLING / BUSINESS LOGIC
Adjutant CloudKitty
TESTING / BENCHMARK
Tempest Patrole
Rally
LIFECYCLE MANAGEMENT
DEPLOYMENT / LIFECYCLE TOOLS
Kolla-Ansible OpenStack-Charms TripleO Bifrost Kayobe
OpenStack-Helm OpenStack-Ansible OpenStack-Chef
PACKAGING RECIPES FOR...
RPM Puppet
Containers (LOCI, Kolla)
Version 2021.02.01
openstack.